Necropolis Casino — წყარო კოდი / Source code

საქპატენტში დეპონირებისთვის · For deposit with Sakpatenti
ვერსია / Version: commit 6a2255b, 9 November 2026 · github.com/POWERLICH21/QuridzeCsharp
ხაზების ნომრები ემთხვევა ორიგინალ ფაილებს; გენერირებული ბლოკები ნაჩვენებია ერთი შენიშვნით. · Line numbers match the original files; generated blocks are shown as one note line.

FILE / ფაილი	CONTENT / შინაარსი	LINES	SHA-256
`index.html`	Single-player game: page, styles, game logic and AI / ერთმოთამაშიანი თამაში	1757	`a68dcf6b8c739f037dfc5f593423dc4d66a9f7d2a6444378888eac8476904f9a`
`multiplayer/server.js`	Two-player game server / ორმოთამაშიანი თამაშის სერვერი	483	`b7d85146abd4000903dbe06d5ba589564e05f6a4aae4f6f2de9143d1d9caa95c`
`multiplayer/client.html`	Two-player game page / ორმოთამაშიანი თამაშის გვერდი	1912	`8240b8799215005fd19e5ba28b2f4fae9deabeff9dcb9bb363b63afc4d7352e9`
`build.py`	Build script: embeds pictures and styles / აწყობის სკრიპტი	71	`55f5dea4855c6b39e75c434cf81a299621b095022c2c30d9262760cdd2d9120e`
`art-source/cut_figures.py`	Art step 1: cuts creatures out of the source images / ილუსტრაციები, ნაბიჯი 1	76	`9b57b4bfb9b02cbe5a1ac61d417861bb90eac2abdcf0a1be206f3dc4c4e77810`
`art-source/make_card_art.py`	Art step 2: draws the neon card pictures / ილუსტრაციები, ნაბიჯი 2	196	`176a362b0bcf52db137c54fc8a719093ceafb24f73ab0fd157ead55bc3d6bedc`
`multiplayer/start-server.bat`	Windows start script / Windows-ის გაშვების სკრიპტი	22	`028d76101b167b99cb7e933b24239d34279fc3f274f37b057a106db623e4ae7f`
`multiplayer/start-server.command`	macOS start script / macOS-ის გაშვების სკრიპტი	6	`b5b86e3a1e3bfcbc34fa95b27c24ed714440b849a51af5d53fe0d3d81cc77751`

index.html

Single-player game: page, styles, game logic and AI / ერთმოთამაშიანი თამაში · 1757 lines · SHA-256 a68dcf6b8c739f037dfc5f593423dc4d66a9f7d2a6444378888eac8476904f9a

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover">
    <title>Necropolis Casino</title>
    <link href="https://fonts.googleapis.com/css2?family=Exo+2:wght@400;500;600;700;800&family=Orbitron:wght@600;700;900&display=swap" rel="stylesheet">
    <style>
        /* Neon Necropolis: bright neon accents on calm, solid dark surfaces.
           Layout: enemy rank on top, wager belt across the middle, your rank below;
           on phones the action buttons sit in a bar at the bottom, within thumb reach. */
        :root {
            color-scheme: dark;
            --bg: #070812;
            --surface: #0f1124;
            --surface-hi: #151833;
            --fg: #e7eaff;
            --muted: #a0a6cc;
            --faint: #646a96;
            --line: rgba(140, 150, 255, 0.16);
            --line-strong: rgba(140, 150, 255, 0.32);
            --card-ink: #0a0b18;
            /* Neon accents, as "r g b" so they can take any alpha */
            --you-rgb: 46 230 255;      /* Your side: cyan */
            --foe-rgb: 255 61 154;      /* The opponent's side: magenta */
            --life-rgb: 77 255 136;     /* Animate Dead: green */
            --void-rgb: 180 107 255;    /* Implosion and the crypt: violet */
            --gold-rgb: 255 216 77;     /* Bets and fortune: gold */
            --danger-rgb: 255 77 109;   /* Resolve and losses: red */
            --you: rgb(var(--you-rgb));
            --foe: rgb(var(--foe-rgb));
            --ally: rgb(var(--life-rgb));
            --enemy: rgb(var(--void-rgb));
            --void: rgb(var(--void-rgb));
            --gold: rgb(var(--gold-rgb));
            --blood: rgb(var(--danger-rgb));
            --font-display: 'Orbitron', 'Exo 2', system-ui, sans-serif;
            --font-ui: 'Exo 2', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
            --font-num: 'Exo 2', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif; /* Plain digits read fastest */
            --radius: 14px;
            --card-radius: 10px;
            --bar-h: 0px;
            /* Card width: five cards plus gaps and gutters always fit the screen */
            --cw: min(130px, calc((100vw - 130px) / 5));
        }

        html { height: 100%; -webkit-text-size-adjust: 100%; }

        body {
            min-height: 100%;
            margin: 0;
            display: flex;
            flex-direction: column;
            font-family: var(--font-ui);
            color: var(--fg);
            background-color: var(--bg);
            /* A faint neon grid under soft violet and cyan light */
            background-image:
                radial-gradient(ellipse 90% 45% at 50% 0%, rgb(var(--void-rgb) / 0.13), transparent 70%),
                radial-gradient(ellipse 90% 45% at 50% 100%, rgb(var(--you-rgb) / 0.09), transparent 70%),
                linear-gradient(rgb(140 150 255 / 0.035) 1px, transparent 1px),
                linear-gradient(90deg, rgb(140 150 255 / 0.035) 1px, transparent 1px);
            background-size: auto, auto, 36px 36px, 36px 36px;
            overflow-x: hidden;
            -webkit-user-select: none;
            user-select: none;
            -webkit-tap-highlight-color: transparent;
        }

        button, input { touch-action: manipulation; }

        .crypt-table {
            flex: 1;
            display: flex;
            flex-direction: column;
            align-items: center;
            gap: 14px;
            padding: calc(18px + env(safe-area-inset-top, 0px)) 16px calc(24px + env(safe-area-inset-bottom, 0px));
        }

```

```
        /* Neon sign title */
        .brand {
            margin: 0;
            font-family: var(--font-display);
            font-weight: 900;
            font-size: clamp(1.05rem, 4.4vw, 1.75rem);
            letter-spacing: 0.2em;
            text-transform: uppercase;
            text-align: center;
            color: #f1e6ff;
            text-shadow: 0 0 4px rgb(var(--void-rgb) / 0.9), 0 0 16px rgb(var(--void-rgb) / 0.55);
        }
        .brand span {
            color: #e6fdff;
            text-shadow: 0 0 4px rgb(var(--you-rgb) / 0.9), 0 0 16px rgb(var(--you-rgb) / 0.55);
        }

        /* Solid panels with a thin neon edge */
        .panel {
            background: linear-gradient(180deg, rgb(21 24 51 / 0.94), rgb(12 14 30 / 0.94));
            border: 1px solid var(--line);
            border-radius: var(--radius);
            box-shadow: 0 10px 30px rgb(0 0 0 / 0.55), inset 0 1px 0 rgb(255 255 255 / 0.04);
        }

        .side {
            width: 100%;
            max-width: 64rem;
            display: flex;
            flex-direction: column;
            align-items: center;
            gap: 10px;
        }
        .side-head {
            width: 100%;
            display: flex;
            align-items: center;
            justify-content: space-between;
            gap: 8px 12px;
            padding-inline: 6px;
        }
        .side-title {
            margin: 0;
            min-width: 0;
            font-family: var(--font-display);
            font-weight: 700;
            font-size: clamp(0.92rem, 3.6vw, 1.35rem);
            letter-spacing: 0.12em;
            text-transform: uppercase;
            overflow-wrap: anywhere;
        }
        .side-foe .side-title { color: var(--foe); text-shadow: 0 0 10px rgb(var(--foe-rgb) / 0.55); }
        .side-you .side-title { color: var(--you); text-shadow: 0 0 10px rgb(var(--you-rgb) / 0.55); }

        .strength-chip {
            flex-shrink: 0;
            padding: 0.3rem 0.85rem;
            border-radius: 999px;
            border: 1px solid var(--line-strong);
            background: rgb(10 11 24 / 0.85);
            color: var(--muted);
            font-weight: 600;
            font-size: clamp(0.82rem, 3.2vw, 1rem);
            white-space: nowrap;
        }
        .strength-chip b {
            margin-left: 0.35em;
            font-family: var(--font-num);
            font-weight: 800;
            color: var(--fg);
            font-variant-numeric: tabular-nums;
        }
        .side-foe .strength-chip { border-color: rgb(var(--foe-rgb) / 0.4); }
        .side-you .strength-chip { border-color: rgb(var(--you-rgb) / 0.4); }

        .board {
            width: 100%;
            display: flex;
            justify-content: center;
            gap: 16px;
            padding: 16px;
        }
        .side-foe .board {
            border-color: rgb(var(--foe-rgb) / 0.32);
            box-shadow: 0 0 22px rgb(var(--foe-rgb) / 0.08), 0 10px 30px rgb(0 0 0 / 0.55);
        }
```

```
        .side-you .board {
            border-color: rgb(var(--you-rgb) / 0.32);
            box-shadow: 0 0 22px rgb(var(--you-rgb) / 0.08), 0 10px 30px rgb(0 0 0 / 0.55);
        }

        /* 3D Card Mechanics */
        .card-slot {
            width: var(--cw);
            height: calc(var(--cw) * 1.385);
            position: relative;
        }

        /* Random tabletop offset lives on the wrapper; animations run on .card-fx so they never erase it */
        .card-wrapper {
            position: absolute;
            inset: 0;
            perspective: 800px;
            border-radius: var(--card-radius);
            transition: transform 0.3s ease, filter 0.3s ease;
        }

        .card-fx {
            position: relative;
            width: 100%;
            height: 100%;
            border-radius: var(--card-radius);
            perspective: 1000px;
            transition: scale 0.2s ease;
        }

        .card {
            width: 100%;
            height: 100%;
            position: relative;
            transition: transform 0.6s cubic-bezier(0.4, 0.0, 0.2, 1);
            transform-style: preserve-3d;
        }

        .card.face-down {
            transform: rotateY(180deg);
        }

        .card-face {
            position: absolute;
            inset: 0;
            -webkit-backface-visibility: hidden;
            backface-visibility: hidden;
            border-radius: var(--card-radius);
            display: flex;
            flex-direction: column;
            justify-content: space-between;
            padding: calc(var(--cw) * 0.06);
            background-color: var(--card-ink);
            box-shadow: 0 6px 14px rgb(0 0 0 / 0.6);
        }

        /* --lv is the card's level colour (set on each card) */
        .card-front {
            --lv: var(--fg);
            isolation: isolate;
            border: 2px solid var(--lv);
            background: linear-gradient(180deg, #121430 0%, var(--card-ink) 100%);
            box-shadow: 0 0 16px -5px var(--lv), 0 6px 14px rgb(0 0 0 / 0.6);
        }

        /* Card back: the crypt's violet neon */
        .card-back {
            background:
                radial-gradient(circle at 50% 50%, rgb(var(--void-rgb) / 0.2), transparent 62%),
                repeating-linear-gradient(45deg, rgb(var(--void-rgb) / 0.08) 0 2px, transparent 2px 9px),
                #0c0b1e;
            border: 2px solid rgb(var(--void-rgb) / 0.6);
            box-shadow: 0 0 14px -5px var(--void), inset 0 0 0 3px #0c0b1e, inset 0 0 0 4px rgb(var(--void-rgb) / 0.25);
            transform: rotateY(180deg);
            align-items: center;
            justify-content: center;
        }

        .card-back::after {
            content: '☥';
            font-family: system-ui, sans-serif;
            font-size: calc(var(--cw) * 0.36);
            color: #dcc4ff;
            text-shadow: 0 0 6px var(--void), 0 0 16px rgb(var(--void-rgb) / 0.6);
        }

```

```
        .card-name {
            position: relative;
            z-index: 2;
            font-size: max(8px, calc(var(--cw) * 0.1));
            line-height: 1.15;
            font-weight: 700;
            letter-spacing: 0.03em;
            text-align: center;
            color: var(--lv);
            text-shadow: 0 1px 2px #000, 0 0 6px var(--lv);
        }

        .card-sigil {
            flex-grow: 1;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: calc(var(--cw) * 0.28);
            color: var(--lv);
            opacity: 0.6;
            text-shadow: 0 0 6px var(--lv);
        }

        .card-foot {
            position: relative;
            z-index: 2;
            display: flex;
            justify-content: center;
        }

        /* Creature art: full-bleed neon picture, shaded at the top and bottom so the name and points stay readable */
        .card-art {
            position: absolute;
            inset: 0;
            width: 100%;
            height: 100%;
            object-fit: cover;
            border-radius: calc(var(--card-radius) - 2px);
            pointer-events: none;
            z-index: 0;
        }

        .card-front.has-art::before {
            content: '';
            position: absolute;
            inset: 0;
            border-radius: calc(var(--card-radius) - 2px);
            background: linear-gradient(180deg,
                rgb(5 6 14 / 0.88) 0%, rgb(5 6 14 / 0) 22%,
                rgb(5 6 14 / 0) 74%, rgb(5 6 14 / 0.7) 100%);
            pointer-events: none;
            z-index: 1;
        }

        .slot-empty {
            width: 100%;
            height: 100%;
            border: 1.5px dashed rgb(140 150 255 / 0.22);
            border-radius: var(--card-radius);
            background: rgb(140 150 255 / 0.025);
        }

        /* Animations */
        /* A card settling onto the table after its flight from the crypt */
        @keyframes land {
            0% { transform: scale(1.08); filter: brightness(1.4); }
            100% { transform: none; filter: none; }
        }

        .animate-land {
            animation: land 0.28s ease-out both;
        }

        /* Card in flight from the crypt pile to its slot */
        .flyer {
            position: fixed;
            z-index: 60;
            pointer-events: none;
            perspective: 800px;
        }
        .flyer .card {
            transition-duration: 0.3s;
        }

        /* The crypt: the deck everyone draws from */
        .deck-pile {
```

```
            position: relative;
            flex-shrink: 0;
            width: 42px;
            height: 58px;
            border-radius: 7px;
            border: 2px solid rgb(var(--void-rgb) / 0.6);
            background:
                repeating-linear-gradient(45deg, rgb(var(--void-rgb) / 0.1) 0 2px, transparent 2px 7px),
                #0c0b1e;
            box-shadow:
                3px -3px 0 -1px #0c0b1e, 3px -3px 0 0 rgb(var(--void-rgb) / 0.45),
                6px -6px 0 -1px #0c0b1e, 6px -6px 0 0 rgb(var(--void-rgb) / 0.3),
                0 0 14px rgb(var(--void-rgb) / 0.25);
            display: flex;
            align-items: center;
            justify-content: center;
            transition: opacity 0.3s;
        }
        .deck-pile::after {
            content: '†';
            font-family: system-ui, sans-serif;
            font-size: 1.5rem;
            color: #dcc4ff;
            text-shadow: 0 0 6px var(--void);
        }
        .deck-pile.empty {
            opacity: 0.3;
        }

        @keyframes spectralFlame {
            0% { transform: scale(1); box-shadow: 0 0 0 rgb(var(--life-rgb) / 0); filter: brightness(1); }
            20% { transform: scale(1.15) rotate(-3deg); box-shadow: 0 0 36px rgb(var(--life-rgb) / 0.8); filter: brightnes
s(1.4); }
            40% { transform: scale(1.15) rotate(3deg); box-shadow: 0 0 56px rgb(var(--life-rgb) / 1); filter: brightness
(1.7); }
            60% { transform: scale(1.15) rotate(-3deg); box-shadow: 0 0 56px rgb(var(--life-rgb) / 1); filter: brightness
(1.7); }
            100% { transform: scale(1); box-shadow: 0 0 0 rgb(var(--life-rgb) / 0); filter: brightness(1); }
        }

        .anim-spectral {
            animation: spectralFlame 1s ease-out forwards;
        }

        @keyframes darkVoid {
            0% { transform: scale(1); filter: contrast(1); }
            30% { transform: scale(0.8) rotate(10deg); filter: contrast(1.6) hue-rotate(20deg); box-shadow: inset 0 0 50px
var(--void), 0 0 40px var(--void); }
            60% { transform: scale(0.1) rotate(-20deg); opacity: 1; }
            100% { transform: scale(0); opacity: 0; }
        }

        .anim-void {
            animation: darkVoid 0.8s ease-in forwards;
        }

        @keyframes bump {
            0% { transform: scale(1); }
            40% { transform: scale(1.35); }
            100% { transform: scale(1); }
        }

        .badge-bump {
            animation: bump 0.35s ease-out;
        }

        /* Targeting Mode: valid targets glow softly (touch screens have no hover) */
        body.targeting {
            cursor: crosshair;
        }
        body.targeting .card-wrapper {
            cursor: crosshair;
        }
        body.targeting #player-slots .card-wrapper {
            filter: drop-shadow(0 0 5px rgb(var(--life-rgb) / 0.55));
        }
        body.targeting #opponent-slots .card-wrapper {
            filter: drop-shadow(0 0 5px rgb(var(--void-rgb) / 0.6));
        }
        body.targeting .card-wrapper:hover .card-fx,
        body.targeting .card-wrapper:focus-visible .card-fx {
            scale: 1.05;
        }
        body.targeting #player-slots .card-wrapper:hover,
        body.targeting #player-slots .card-wrapper:focus-visible {
            filter: drop-shadow(0 0 12px var(--ally));
```

```
        }
        body.targeting #opponent-slots .card-wrapper:hover,
        body.targeting #opponent-slots .card-wrapper:focus-visible {
            filter: drop-shadow(0 0 12px var(--enemy));
        }

        .card-wrapper:focus-visible {
            outline: 2px solid var(--fg);
            outline-offset: 4px;
        }

        /* Matching cards joined into one pile: card edges peek out behind it */
        .card-wrapper::before,
        .card-wrapper::after {
            content: '';
            position: absolute;
            inset: 0;
            border-radius: var(--card-radius);
            border: 2px solid rgb(140 150 255 / 0.4);
            background: var(--card-ink);
            opacity: 0;
            transition: opacity 0.2s;
            pointer-events: none;
        }
        .card-wrapper::before { transform: translate(4px, -4px); z-index: -1; }
        .card-wrapper::after { transform: translate(8px, -8px); z-index: -2; }
        .card-wrapper.stack-2::before,
        .card-wrapper.stack-3::before,
        .card-wrapper.stack-3::after { opacity: 1; }
        .card-wrapper:has(.anim-void)::before,
        .card-wrapper:has(.anim-void)::after { opacity: 0; }

        /* How many matching cards are in the pile (shown from x2 up) */
        .card-count {
            position: absolute;
            top: calc(var(--cw) * -0.12);
            right: calc(var(--cw) * -0.12);
            z-index: 5;
            min-width: max(22px, calc(var(--cw) * 0.25));
            height: max(22px, calc(var(--cw) * 0.25));
            padding: 0 0.3em;
            border-radius: 999px;
            background: #0a0b18;
            border: 2px solid var(--fg);
            color: #fff;
            font-family: var(--font-num);
            font-weight: 800;
            font-size: max(11px, calc(var(--cw) * 0.11));
            font-variant-numeric: tabular-nums;
            display: flex;
            align-items: center;
            justify-content: center;
            box-shadow: 0 0 8px rgb(231 234 255 / 0.45), 0 2px 6px rgb(0 0 0 / 0.6);
        }
        .card-count[hidden] {
            display: none;
        }

        /* Points plaque: what the card gives; its colour is the creature's level */
        .card-points {
            display: inline-flex;
            align-items: baseline;
            gap: 0.25em;
            background: rgb(5 6 14 / 0.86);
            border: 1px solid var(--lv);
            border-radius: 999px;
            padding: 0.12em 0.6em;
            color: var(--lv);
            font-family: var(--font-num);
            font-size: max(12px, calc(var(--cw) * 0.15));
            font-weight: 800;
            line-height: 1.15;
            font-variant-numeric: tabular-nums;
            white-space: nowrap;
            text-shadow: 0 0 6px var(--lv);
            box-shadow: 0 0 10px -3px var(--lv);
        }
        .card-points small {
            font-family: var(--font-ui);
            font-size: 0.6em;
            letter-spacing: 0.08em;
            text-transform: uppercase;
        }

        /* Wager belt */
        .belt {
```

```
            width: 100%;
            max-width: 64rem;
            position: relative;
            z-index: 20;
            display: flex;
            flex-wrap: wrap;
            align-items: center;
            justify-content: space-between;
            gap: 14px 18px;
            padding: 14px 18px;
            border-color: rgb(var(--void-rgb) / 0.3);
        }
        .belt-info {
            display: flex;
            align-items: center;
            gap: 14px;
        }
        .round-box {
            min-width: 5.5rem;
            text-align: center;
        }
        .label {
            font-size: 0.72rem;
            font-weight: 600;
            letter-spacing: 0.14em;
            text-transform: uppercase;
            color: var(--muted);
        }
        .round-value {
            font-family: var(--font-num);
            font-weight: 800;
            font-size: 1.8rem;
            line-height: 1.15;
            color: var(--fg);
            text-shadow: 0 0 10px rgb(var(--void-rgb) / 0.6);
            font-variant-numeric: tabular-nums;
        }
        .crypt-count {
            font-size: 0.76rem;
            color: var(--muted);
        }
        .crypt-count span { font-variant-numeric: tabular-nums; }
        .divider {
            width: 1px;
            align-self: stretch;
            background: var(--line-strong);
        }
        .wager {
            display: flex;
            flex-direction: column;
            gap: 6px;
        }
        .bet-input {
            width: 9.5rem;
            padding: 0.45rem 0.75rem;
            border-radius: 10px;
            border: 1px solid rgb(var(--gold-rgb) / 0.4);
            background: #070815;
            color: var(--fg);
            font-family: var(--font-num);
            font-weight: 700;
            font-size: 1.05rem;
            transition: border-color 0.2s, box-shadow 0.2s;
        }
        .bet-input::placeholder {
            color: var(--faint);
            font-family: var(--font-ui);
            font-weight: 500;
        }
        .bet-input:focus {
            outline: none;
            border-color: var(--gold);
            box-shadow: 0 0 0 3px rgb(var(--gold-rgb) / 0.16), 0 0 14px rgb(var(--gold-rgb) / 0.3);
        }
        .bet-input:disabled { opacity: 0.55; }
        .wager-stats {
            display: flex;
            flex-wrap: wrap;
            gap: 2px 14px;
            font-size: 0.9rem;
            color: var(--muted);
        }
        .wager-stats b {
            font-family: var(--font-num);
            font-weight: 800;
            font-variant-numeric: tabular-nums;
```

```
        }
        .stat-bet b { color: var(--gold); text-shadow: 0 0 8px rgb(var(--gold-rgb) / 0.45); }
        .stat-gold b { color: var(--fg); }

        .action-bar {
            display: flex;
            flex-wrap: wrap;
            justify-content: center;
            gap: 10px;
        }

        /* Neon buttons: --tone is the button's colour */
        .btn-neon {
            --tone: var(--you-rgb);
            min-height: 46px;
            padding: 0.6rem 1.2rem;
            border-radius: 12px;
            border: 1.5px solid rgb(var(--tone) / 0.75);
            background: rgb(10 11 24 / 0.9);
            color: rgb(var(--tone));
            font-family: var(--font-display);
            font-weight: 700;
            font-size: 0.82rem;
            letter-spacing: 0.1em;
            line-height: 1.2;
            text-transform: uppercase;
            text-shadow: 0 0 8px rgb(var(--tone) / 0.5);
            box-shadow: 0 0 14px rgb(var(--tone) / 0.2), inset 0 0 14px rgb(var(--tone) / 0.08);
            transition: background 0.2s, box-shadow 0.2s, opacity 0.2s, transform 0.1s;
        }
        .btn-neon:hover:not(:disabled) {
            background: rgb(var(--tone) / 0.12);
            box-shadow: 0 0 22px rgb(var(--tone) / 0.35), inset 0 0 16px rgb(var(--tone) / 0.14);
        }
        .btn-neon:active:not(:disabled) { transform: translateY(1px); }
        .btn-neon:focus-visible {
            outline: 2px solid rgb(var(--tone));
            outline-offset: 3px;
        }
        .btn-neon:disabled {
            opacity: 0.35;
            cursor: not-allowed;
            box-shadow: none;
            text-shadow: none;
        }
        .btn-main { --tone: var(--you-rgb); background: rgb(var(--you-rgb) / 0.1); }
        .btn-life { --tone: var(--life-rgb); }
        .btn-danger { --tone: var(--danger-rgb); }

        /* How to play: tucked away until wanted */
        .rules {
            width: 100%;
            max-width: 42rem;
            border: 1px solid var(--line);
            border-radius: 12px;
            background: rgb(12 14 30 / 0.75);
        }
        .rules summary {
            display: flex;
            align-items: center;
            justify-content: space-between;
            min-height: 44px;
            padding: 0.6rem 1rem;
            cursor: pointer;
            list-style: none;
            font-family: var(--font-display);
            font-weight: 700;
            font-size: 0.75rem;
            letter-spacing: 0.14em;
            text-transform: uppercase;
            color: var(--muted);
        }
        .rules summary::-webkit-details-marker { display: none; }
        .rules summary::after {
            content: '+';
            font-size: 1.15rem;
            color: var(--you);
            text-shadow: 0 0 8px rgb(var(--you-rgb) / 0.6);
        }
        .rules[open] summary::after { content: '−'; }
        .rules summary:focus-visible {
            outline: 2px solid var(--you);
            outline-offset: -2px;
            border-radius: 12px;
        }
        .rules p {
```

```
            margin: 0;
            padding: 0 1rem 1rem;
            color: var(--muted);
            font-size: 0.92rem;
            line-height: 1.6;
            text-wrap: pretty;
        }
        .hl-life { color: var(--ally); }
        .hl-void { color: var(--enemy); }
        .hl-gold { color: var(--gold); }

        /* Colour key under the rules */
        .level-legend {
            display: flex;
            flex-wrap: wrap;
            justify-content: center;
            gap: 0.4rem 1rem;
            margin: 0;
            padding: 0 0.5rem;
            list-style: none;
            font-size: 0.8rem;
            color: var(--muted);
        }
        .level-legend li {
            display: inline-flex;
            align-items: center;
            gap: 0.4rem;
            white-space: nowrap;
        }
        .level-legend b {
            font-family: var(--font-num);
            font-weight: 800;
            font-variant-numeric: tabular-nums;
            color: var(--lv);
            text-shadow: 0 0 6px var(--lv);
        }
        .legend-swatch {
            width: 0.7rem;
            height: 0.7rem;
            border-radius: 3px;
            background: var(--lv);
            box-shadow: 0 0 8px var(--lv);
        }

        #toast {
            position: fixed;
            left: 50%;
            bottom: calc(var(--bar-h) + 16px + env(safe-area-inset-bottom, 0px));
            z-index: 50;
            transform: translateX(-50%);
            width: max-content;
            max-width: min(32rem, calc(100vw - 32px));
            padding: 0.7rem 1.15rem;
            border-radius: 12px;
            border: 1px solid rgb(var(--you-rgb) / 0.45);
            background: rgb(15 17 36 / 0.97);
            box-shadow: 0 0 18px rgb(var(--you-rgb) / 0.16), 0 10px 30px rgb(0 0 0 / 0.6);
            color: var(--fg);
            font-weight: 600;
            font-size: 1rem;
            text-align: center;
            text-wrap: balance;
            opacity: 0;
            pointer-events: none;
            transition: opacity 0.3s;
        }

        /* Crypt Fortune: the luck roll after a win */
        .fortune {
            position: fixed;
            inset: 0;
            z-index: 70;
            display: flex;
            align-items: center;
            justify-content: center;
            padding: 16px;
            background: rgb(4 5 12 / 0.78);
        }
        .fortune[hidden] {
            display: none;
        }
        .fortune-panel {
            width: min(380px, 100%);
            padding: 1.5rem 1.25rem;
            text-align: center;
            border-color: rgb(var(--void-rgb) / 0.4);
```

```
            box-shadow: 0 0 30px rgb(var(--void-rgb) / 0.18), 0 10px 30px rgb(0 0 0 / 0.6);
        }
        .fortune-title {
            font-family: var(--font-display);
            font-size: 0.75rem;
            font-weight: 700;
            letter-spacing: 0.22em;
            text-transform: uppercase;
            color: var(--muted);
        }
        .fortune-reel {
            width: 7.5rem;
            height: 4.5rem;
            margin: 1rem auto;
            display: flex;
            align-items: center;
            justify-content: center;
            border-radius: 12px;
            border: 2px solid var(--line-strong);
            background: #05060e;
            color: var(--fg);
            font-family: var(--font-num);
            font-size: 2.5rem;
            font-weight: 800;
            font-variant-numeric: tabular-nums;
            transition: color 0.3s, border-color 0.3s, box-shadow 0.3s;
        }
        .fortune-headline {
            min-height: 1.4em;
            font-family: var(--font-display);
            font-size: 1.45rem;
            font-weight: 900;
            letter-spacing: 0.08em;
            text-wrap: balance;
        }
        .fortune-result {
            min-height: 2.8em;
            margin-top: 0.4rem;
            color: var(--fg);
            text-wrap: balance;
        }
        .fortune.tier-lucky .fortune-reel { color: var(--ally); border-color: var(--ally); box-shadow: 0 0 24px rgb(var(--life-rgb) / 0.55); }
        .fortune.tier-lucky .fortune-headline { color: var(--ally); text-shadow: 0 0 12px rgb(var(--life-rgb) / 0.75); }
        .fortune.tier-big .fortune-reel { color: var(--gold); border-color: var(--gold); box-shadow: 0 0 34px rgb(var(--gold-rgb) / 0.7); }
        .fortune.tier-big .fortune-headline { color: var(--gold); text-shadow: 0 0 16px rgb(var(--gold-rgb) / 0.85); }
        .fortune.tier-mega .fortune-reel { color: var(--foe); border-color: var(--foe); box-shadow: 0 0 44px rgb(var(--foe-rgb) / 0.8), 0 0 90px rgb(var(--void-rgb) / 0.6); }
        .fortune.tier-mega .fortune-headline { color: var(--foe); text-shadow: 0 0 18px rgb(var(--foe-rgb) / 0.85), 0 0 36px rgb(var(--void-rgb) / 0.85); }
        .fortune.tier-big .fortune-panel,
        .fortune.tier-mega .fortune-panel {
            animation: fortunePulse 0.9s ease-in-out 3;
        }
        @keyframes fortunePulse {
            0%, 100% { transform: scale(1); }
            50% { transform: scale(1.06); }
        }

        /* Phones: tighter edges, bigger cards, buttons in a bar at the bottom */
        @media (max-width: 640px) {
            :root {
                --cw: calc((100vw - 62px) / 5);
                --bar-h: 68px;
                --radius: 12px;
                --card-radius: 8px;
            }
            .crypt-table {
                gap: 10px;
                padding: calc(12px + env(safe-area-inset-top, 0px)) 10px calc(var(--bar-h) + 18px + env(safe-area-inset-bottom, 0px));
            }
            .side { gap: 6px; }
            .side-head { padding-inline: 2px; }
            .board { gap: 6px; padding: 9px 6px; }
            .belt { padding: 10px 12px; gap: 10px; }
            .belt-info { width: 100%; justify-content: space-between; gap: 10px; }
            .deck-pile { width: 34px; height: 47px; }
            .deck-pile::after { font-size: 1.2rem; }
            .round-box { min-width: 0; }
            .round-value { font-size: 1.35rem; }
            .bet-input { width: 100%; max-width: 9rem; font-size: 16px; } /* 16px stops iOS zooming in */
            .wager-stats { font-size: 0.82rem; gap: 0 12px; }
            .card-points small { display: none; } /* Phone cards are narrow: the number alone */
            .action-bar {
```

```
                position: fixed;
                left: 0;
                right: 0;
                bottom: 0;
                z-index: 40;
                flex-wrap: nowrap;
                gap: 8px;
                padding: 10px 10px calc(10px + env(safe-area-inset-bottom, 0px));
                background: rgb(7 8 18 / 0.95);
                border-top: 1px solid rgb(var(--void-rgb) / 0.32);
                box-shadow: 0 -10px 30px rgb(0 0 0 / 0.6);
                -webkit-backdrop-filter: blur(10px);
                backdrop-filter: blur(10px);
            }
            .action-bar .btn-neon {
                flex: 1 1 0;
                min-width: 0;
                min-height: 48px;
                padding: 0.45rem 0.35rem;
                font-size: 0.66rem;
                letter-spacing: 0.06em;
            }
            #toast { font-size: 0.95rem; }
            .card-count { top: -11px; right: -8px; } /* Clear of the card name on narrow cards */
            .rules p { font-size: 0.9rem; }
            .level-legend { font-size: 0.74rem; gap: 0.35rem 0.8rem; }
        }

        /* Phones held sideways: cards sized by the screen's height, so both armies and the buttons fit */
        @media (orientation: landscape) and (max-height: 520px) {
            :root {
                --cw: min(110px, calc((100vh - 190px) / 2.77), calc((100vw - 120px) / 5));
                --bar-h: 0px;
            }
            @supports (height: 100dvh) {
                :root { --cw: min(110px, calc((100dvh - 190px) / 2.77), calc((100vw - 120px) / 5)); }
            }
            .brand { display: none; }
            .crypt-table {
                gap: 8px;
                padding: 8px max(16px, env(safe-area-inset-right, 0px)) calc(12px + env(safe-area-inset-bottom, 0px)) max
(16px, env(safe-area-inset-left, 0px));
            }
            .side { gap: 4px; }
            .side-title { font-size: 0.9rem; }
            .strength-chip { padding: 0.15rem 0.7rem; font-size: 0.82rem; }
            .board { gap: 10px; padding: 6px; }
            .belt { padding: 6px 12px; gap: 8px 14px; }
            .belt-info { width: auto; gap: 12px; }
            .deck-pile { width: 30px; height: 41px; }
            .deck-pile::after { font-size: 1.05rem; }
            .round-value { font-size: 1.2rem; }
            .label, .crypt-count { font-size: 0.68rem; }
            .bet-input { width: 7.5rem; padding: 0.3rem 0.6rem; }
            .wager { gap: 3px; }
            .wager-stats { font-size: 0.8rem; }
            .action-bar {
                position: static;
                flex-wrap: nowrap;
                gap: 8px;
                padding: 0;
                background: none;
                border: 0;
                box-shadow: none;
                -webkit-backdrop-filter: none;
                backdrop-filter: none;
            }
            .action-bar .btn-neon {
                flex: 0 1 auto;
                min-height: 40px;
                padding: 0.4rem 0.8rem;
                font-size: 0.68rem;
                letter-spacing: 0.06em;
            }
            .card-points small { display: none; }
            #toast { top: 8px; bottom: auto; } /* Over the opponent's title, clear of your own cards */
        }

        @media (prefers-reduced-motion: reduce) {
            .card, .card-wrapper, .card-fx { transition-duration: 0.01ms !important; }
            .animate-land, .anim-spectral, .anim-void, .badge-bump, .fortune-panel {
                animation-duration: 0.01ms !important;
                animation-delay: 0ms !important;
            }
        }
    </style>
```

```
    <!-- build:tailwind (precompiled Tailwind CSS, generated by build.py) -->
```
[lines 932–932 omitted: generated Tailwind CSS (third-party, MIT licence) / გენერირებული Tailwind CSS]
```
    <!-- /build:tailwind -->
</head>
<body>
<main class="crypt-table">

    <h1 class="brand">Necropolis <span>Casino</span></h1>

    <!-- Opponent Board (Top) -->
    <section class="side side-foe" aria-label="Opponent board">
        <div class="side-head">
            <h2 class="side-title">Opponent (AI)</h2>
            <div class="strength-chip">Strength <b id="opp-strength">???</b></div>
        </div>
        <div id="opponent-slots" class="board panel">
            <!-- Opponent Cards Injected Here -->
        </div>
    </section>

    <!-- Middle Belt (Controls & Decks) -->
    <section class="belt panel" aria-label="Wagers and actions">
        <div class="belt-info">
            <div id="deck-pile" class="deck-pile" role="img" aria-label="The crypt: the deck both armies draw from"></div>
            <div class="round-box">
                <div class="label">Round</div>
                <div id="round-display" class="round-value">1 / 3</div>
                <div class="crypt-count"><span id="deck-display">59</span> in the crypt</div>
            </div>
            <div class="divider"></div>
            <div class="wager">
                <label for="bet-input" class="sr-only">Bet amount in gold</label>
                <input type="number" id="bet-input" class="bet-input" min="0" step="1" inputmode="numeric" placeholder="Be
t amount">
                <div class="wager-stats">
                    <span class="stat-bet">Your bet <b id="pot-display">0</b></span>
                    <span class="stat-gold">Gold <b id="gold-display">1000</b></span>
                </div>
            </div>
        </div>

        <div class="action-bar">
            <button id="btn-draw" class="btn-neon btn-main">Draw Army</button>
            <button id="btn-spell" class="btn-neon btn-life" disabled>Cast Spell (1)</button>
            <button id="btn-resolve" class="btn-neon btn-danger" disabled>Resolve Round</button>
        </div>
    </section>

    <!-- Player Board (Bottom) -->
    <section class="side side-you" aria-label="Your board">
        <div id="player-slots" class="board panel">
            <!-- Player Cards Injected Here -->
        </div>
        <div class="side-head">
            <h2 class="side-title">Your Army</h2>
            <div class="strength-chip">Strength <b id="player-strength">0</b></div>
        </div>
    </section>

    <details class="rules">
        <summary>How to play &amp;amp; payouts</summary>
        <p>
            Bet, then both armies draw five cards from the crypt, dealt in pairs of similar value. Each card shows the poi
nts it gives, and its colour shows its level.
            Matching cards join into one pile: the x2, x3 badge shows how many, and their points add up. Each round you ge
t one spell:
            <span class="hl-life">Animate Dead</span> doubles one of your cards and upgrades it,
            <span class="hl-void">Implosion</span> destroys an enemy card. The AI casts its spell after yours.
            Tip: the AI hunts your biggest card, so destroying its biggest card is usually the safe play.
        </p>
        <p>
            You bet once, before round 1. If your army is stronger after three rounds, you get your bet back plus winnings
of bet &amp;times; 0.20 &amp;times; (your points &amp;divide; 100); a draw returns your bet.
            After every win the <span class="hl-gold">Crypt Fortune</span> may multiply your winnings: Lucky Win &amp;times;2
(1 in 50 wins), BIG WIN &amp;times;10 (1 in 1,000), MEGA WIN &amp;times;50 (1 in 20,000).
        </p>
    </details>
    <ul id="level-legend" class="level-legend" aria-label="Card colour for each level"></ul>

</main>

    <!-- Crypt Fortune: luck roll after a win -->
    <div id="fortune" class="fortune" hidden>
        <div class="fortune-panel panel" role="status" aria-live="polite">
            <div class="fortune-title">Crypt Fortune</div>
            <div id="fortune-reel" class="fortune-reel">×1</div>
```

```
            <div id="fortune-headline" class="fortune-headline"></div>
            <div id="fortune-result" class="fortune-result"></div>
        </div>
    </div>

    <!-- Toast Notification -->
    <div id="toast" role="status" aria-live="polite">
        Notification
    </div>

    <!-- build:card-art (pictures from img/ as data URIs, generated by build.py) -->
```
[lines 1023–1040 omitted: 14 embedded card pictures (WebP, base64) – the same files as img/.webp / ჩაშენებული ბარათების სურათები]*
```
    <!-- /build:card-art -->

    <script>
        // --- GAME DATA & CONFIG ---
        // One entry per creature level; its colour marks the level on the cards.
        // A card spawns `spawn` creatures of `power` each, so it gives spawn x power points (20 up to 60).
        // count: copies of the card in the deck.
        // art: card picture (img/<art>.webp is the weaker base unit; img/<art>-empowered.webp is the upgrade from Animate Dead)
        const UNIT_DATA = [
            { name: 'Skeleton', level: 1, spawn: 20, power: 1, count: 20, color: '#cfd8ff', art: 'skeleton' },       // Ice white: 20 pts
            { name: 'Zombie', level: 2, spawn: 12, power: 2, count: 12, color: '#4dff88', art: 'zombie' },          // Green: 24 pts
            { name: 'Wight', level: 3, spawn: 9, power: 3, count: 9, color: '#38d6ff', art: 'wight' },              // Sky blue: 27 pts
            { name: 'Vampire', level: 4, spawn: 8, power: 4, count: 8, color: '#c77dff', art: 'vampire' },          // Violet: 32 pts
            { name: 'Lich', level: 5, spawn: 6, power: 7, count: 6, color: '#ff9a3c', art: 'lich' },                // Orange: 42 pts
            { name: 'Dark Knight', level: 6, spawn: 3, power: 17, count: 3, color: '#ff4d6d', art: 'dark-knight' }, // Red: 51 pts
            { name: 'Bone Dragon', level: 7, spawn: 1, power: 60, count: 1, color: '#ffe047', art: 'bone-dragon' }  // Gold: 60 pts
        ];
        UNIT_DATA.forEach(unit => { unit.pts = unit.spawn * unit.power; }); // Points one card gives

        function artSrc(unit, empowered) {
            // Embedded copy first; falls back to the img/ file if the page hasn't been built
            const name = `${unit.art}${empowered ? '-empowered' : ''}`;
            return CARD_ART[name] || `img/${name}.webp`;
        }

        // Decode the pictures up front so dealt cards and empowered swaps never show up blank
        UNIT_DATA.filter(unit => unit.art).forEach(unit => {
            new Image().src = artSrc(unit, false);
            new Image().src = artSrc(unit, true);
        });

        const SLOT_COUNT = 5;
        const CARDS_PER_ROUND = 5; // Each army draws this many cards every round
        const PARTNER_RANGE = 10;  // A dealt pair differs by at most this many points when the crypt allows
        const AI_SMART_CHANCE = 0.85; // How often the AI plays its best spell; otherwise it casts at random.
                                      // Tuned so even the strongest strategy found returns about 96% (RTP)
        const TOTAL_ROUNDS = 3;
        const STARTING_GOLD = 1000;
        const DEFAULT_BET = 100;
        const GOLD_KEY = 'necropolis-casino-gold';

        let deck = [];
        let playerArmy = [];
        let opponentArmy = [];

        let currentRound = 1;
        let totalPot = 0;
        let phase = 'BETTING'; // BETTING, ACTION, RESOLVED
        let fogOfWar = true;
        let spellAvailable = false;
        let isTargeting = false;
        let busy = false; // true while a spell animation plays
        let gold = loadGold();

        const REDUCE_MOTION = window.matchMedia?.('(prefers-reduced-motion: reduce)').matches ?? false;
        const FLIGHT_MS = REDUCE_MOTION ? 0 : 340;       // One card flying from the crypt to the table
        const DEAL_STAGGER_MS = REDUCE_MOTION ? 0 : 130; // The AI's card leaves just after yours

        // --- DOM ELEMENTS ---
        const els = {
            oppSlots: document.getElementById('opponent-slots'),
            playerSlots: document.getElementById('player-slots'),
            oppStrength: document.getElementById('opp-strength'),
            playerStrength: document.getElementById('player-strength'),
            btnDraw: document.getElementById('btn-draw'),
```

```
            btnSpell: document.getElementById('btn-spell'),
            btnResolve: document.getElementById('btn-resolve'),
            roundDisp: document.getElementById('round-display'),
            potDisp: document.getElementById('pot-display'),
            goldDisp: document.getElementById('gold-display'),
            deckDisp: document.getElementById('deck-display'),
            deckPile: document.getElementById('deck-pile'),
            betInput: document.getElementById('bet-input'),
            toast: document.getElementById('toast'),
            fortune: document.getElementById('fortune'),
            fortuneReel: document.getElementById('fortune-reel'),
            fortuneHeadline: document.getElementById('fortune-headline'),
            fortuneResult: document.getElementById('fortune-result'),
            legend: document.getElementById('level-legend'),
            body: document.body
        };

        // Colour key: which colour is which level, and the points one card gives
        els.legend.innerHTML = UNIT_DATA.map(unit =>
            `<li style="--lv: ${unit.color};"><span class="legend-swatch"></span>Lv ${unit.level} ${unit.name} <b>${unit.p
ts}</b></li>`
        ).join('');

        // --- STORAGE (gold survives a page reload; works without it too) ---
        function loadGold() {
            try {
                const saved = parseInt(localStorage.getItem(GOLD_KEY), 10);
                return Number.isFinite(saved) && saved > 0 ? saved : STARTING_GOLD;
            } catch (e) {
                return STARTING_GOLD;
            }
        }

        function saveGold() {
            try {
                localStorage.setItem(GOLD_KEY, String(gold));
            } catch (e) {
                // Storage blocked: gold just won't persist across reloads
            }
        }

        // --- CORE LOGIC ---
        function initGame() {
            buildDeck();
            currentRound = 1;
            totalPot = 0;
            playerArmy = [];
            opponentArmy = [];
            phase = 'BETTING';
            fogOfWar = true;
            spellAvailable = false;
            busy = false;
            setTargeting(false);

            if (gold <= 0) {
                gold = STARTING_GOLD;
                saveGold();
                showToast(`Your purse is empty. The Necropolis lends you ${STARTING_GOLD} gold.`);
            } else {
                showToast("Welcome to the Necropolis. Place your bet to begin.");
            }
            els.betInput.value = Math.min(DEFAULT_BET, gold);
            updateUI();
        }

        function buildDeck() {
            deck = [];
            UNIT_DATA.forEach(unit => {
                for (let i = 0; i < unit.count; i++) {
                    deck.push({ ...unit, id: Math.random().toString(36).slice(2, 11) });
                }
            });
            // Shuffle
            for (let i = deck.length - 1; i > 0; i--) {
                const j = Math.floor(Math.random() * (i + 1));
                [deck[i], deck[j]] = [deck[j], deck[i]];
            }
        }

        let toastTimer = null;
        function showToast(msg, duration = 3000) {
            clearTimeout(toastTimer);
            els.toast.textContent = msg;
            els.toast.style.opacity = '1';
            toastTimer = setTimeout(() => { els.toast.style.opacity = '0'; }, duration);
        }
```

```
1191
1192        // Points a card on the table gives: matching cards add up, Animate Dead doubles them
1193        function stackPoints(stack) {
1194            return stack.pts * stack.qty;
1195        }
1196
1197        function getArmyStrength(army) {
1198            return army.reduce((total, stack) => total + stackPoints(stack), 0);
1199        }
1200
1201        function wait(ms) {
1202            return new Promise(resolve => setTimeout(resolve, ms));
1203        }
1204
1205        // --- ACTIONS ---
1206        els.btnDraw.addEventListener('click', async () => {
1207            if (phase !== 'BETTING' || busy) return;
1208
1209            // The bet is placed once, before round 1, and stays locked for the whole game
1210            if (currentRound === 1) {
1211                const bet = Number(els.betInput.value.trim());
1212                if (!Number.isInteger(bet) || bet < 1) {
1213                    showToast("Place a bet of at least 1 gold to begin.");
1214                    return;
1215                }
1216                if (bet > gold) {
1217                    showToast(`You only have ${gold} gold.`);
1218                    return;
1219                }
1220                gold -= bet;
1221                saveGold();
1222                totalPot = bet;
1223                els.betInput.value = bet;
1224            }
1225            phase = 'ACTION';
1226            fogOfWar = true;
1227            spellAvailable = true;
1228
1229            // Deal the cards one at a time, alternating, so both armies draw the same number
1230            busy = true;
1231            updateUI();
1232            await dealArmies(CARDS_PER_ROUND);
1233            busy = false;
1234            updateUI();
1235            showToast("Armies Drawn! Cast a spell or Resolve.");
1236        });
1237
1238        els.betInput.addEventListener('keydown', (e) => {
1239            if (e.key === 'Enter') els.btnDraw.click();
1240        });
1241
1242        // Cards are dealt in matched pairs so both armies receive very similar points: one side (picked at
1243        // random) draws a random card, then the other side gets a random card worth about the same
1244        async function dealArmies(cardsEach) {
1245            for (let i = 0; i < cardsEach; i++) {
1246                const playerLeads = Math.random() < 0.5;
1247                const lead = drawLeadCard(playerLeads ? playerArmy : opponentArmy);
1248                const partner = lead.landing && drawPartnerCard(playerLeads ? opponentArmy : playerArmy, lead.landing.car
    d);
1249                els.deckDisp.textContent = deck.length;
1250                els.deckPile.classList.toggle('empty', deck.length === 0);
1251
1252                // Cards with nowhere to go fly off first, then the pair flies out together
1253                for (const card of lead.discards) {
1254                    await flyCard(card, !playerLeads, null);
1255                    if (playerLeads) showToast(`No room for a ${card.name}: it crumbles to dust.`, 1800);
1256                }
1257                if (!lead.landing) return; // The crypt is empty
1258                await Promise.all([
1259                    landCard(lead.landing, !playerLeads),
1260                    partner ? wait(DEAL_STAGGER_MS).then(() => landCard(partner, playerLeads)) : null
1261                ]);
1262            }
1263        }
1264
1265        // Where a card would go: onto its matching pile, or into the first free slot (null if neither)
1266        function findPlacement(army, card) {
1267            const existing = army.find(c => c.name === card.name && c.qty > 0);
1268            if (existing) return { card, army, index: army.indexOf(existing), existing };
1269            const freeIndex = findFreeSlot(army);
1270            return freeIndex === -1 ? null : { card, army, index: freeIndex, existing: null };
1271        }
1272
1273        // The leading side draws random cards until one fits; a card with no matching pile and no free slot is discarded
1274        function drawLeadCard(army) {
1275            const discards = [];
```

```
            while (deck.length > 0) {
                const card = deck.pop();
                const landing = findPlacement(army, card);
                if (landing) return { discards, landing };
                discards.push(card);
            }
            return { discards, landing: null };
        }

        // The other side gets a random card that fits and is worth within PARTNER_RANGE points of the
        // leader's card (or the closest in value when none is that close)
        function drawPartnerCard(army, leadCard) {
            const fits = deck.map((card, i) => i).filter(i => findPlacement(army, deck[i]));
            if (fits.length === 0) return null;
            const gap = i => Math.abs(deck[i].pts - leadCard.pts);
            let pool = fits.filter(i => gap(i) <= PARTNER_RANGE);
            if (pool.length === 0) {
                const closest = Math.min(...fits.map(gap));
                pool = fits.filter(i => gap(i) === closest);
            }
            const [card] = deck.splice(pool[Math.floor(Math.random() * pool.length)], 1);
            return findPlacement(army, card);
        }

        // Flies a card to its spot on the table and adds it to the army
        async function landCard(placement, isOpponent) {
            const { card, army, index, existing } = placement;
            if (!existing) {
                // Random placement values for the tabletop feel
                card.qty = 1;
                card.rot = (Math.random() * 12) - 6; // -6deg to +6deg
                card.tx = (Math.random() * 10) - 5;  // -5px to +5px
                card.ty = (Math.random() * 10) - 5;  // -5px to +5px
            }
            const pile = existing || card;
            await flyCard(card, isOpponent, { index, rot: pile.rot, tx: pile.tx, ty: pile.ty });
            if (existing) {
                existing.qty += 1;
            } else {
                army[index] = card;
            }
            updateUI();
        }

        // Animates one card from the crypt pile to a slot (or off the table when it is discarded)
        function flyCard(card, isOpponent, target) {
            const slots = isOpponent ? els.oppSlots : els.playerSlots;
            const slotRect = slots.children[target ? target.index : Math.floor(SLOT_COUNT / 2)].getBoundingClientRect();
            const pileRect = els.deckPile.getBoundingClientRect();
            const left = pileRect.left + pileRect.width / 2 - slotRect.width / 2;
            const top = pileRect.top + pileRect.height / 2 - slotRect.height / 2;

            const flyer = document.createElement('div');
            flyer.className = 'flyer';
            flyer.style.cssText = `left: ${left}px; top: ${top}px; width: ${slotRect.width}px; height: ${slotRect.height}p
x;`;
            // Your cards turn face up on the way; the AI's stay hidden
            flyer.innerHTML = `<div class="card face-down">${isOpponent ? '' : cardFrontHTML({ ...card, qty: 1 })}<div cla
ss="card-face card-back"></div></div>`;
            document.body.appendChild(flyer);
            if (!isOpponent) {
                requestAnimationFrame(() => flyer.querySelector('.card').classList.remove('face-down'));
            }

            const dx = slotRect.left - left;
            const dy = slotRect.top - top;
            const from = `translate(0, 0) scale(${pileRect.width / slotRect.width}) rotate(-12deg)`;
            const frames = target
                ? [{ transform: from }, { transform: `translate(${dx + target.tx}px, ${dy + target.ty}px) scale(1) rotate
(${target.rot}deg)` }]
                : [{ transform: from, opacity: 1 },
                   { transform: `translate(${dx}px, ${dy}px) scale(0.9) rotate(6deg)`, opacity: 1, offset: 0.65 },
                   { transform: `translate(${dx}px, ${dy - 30}px) scale(0.4) rotate(24deg)`, opacity: 0 }];
            const flight = flyer.animate(frames, {
                duration: target ? FLIGHT_MS : FLIGHT_MS * 2,
                easing: 'cubic-bezier(0.25, 0.8, 0.3, 1)'
            });
            return flight.finished.catch(() => {}).then(() => flyer.remove());
        }

        // A destroyed stack frees its slot for new recruits
        function findFreeSlot(army) {
            for (let i = 0; i < SLOT_COUNT; i++) {
                if (!army[i] || army[i].qty === 0) return i;
            }
            return -1;
```

```
        }

        // --- SPELL SYSTEM (PLAYER) ---
        function spellLabel() {
            return isTargeting ? "Cancel Target" : `Cast Spell (${spellAvailable ? 1 : 0})`;
        }

        function setTargeting(on) {
            isTargeting = on;
            els.body.classList.toggle('targeting', on);
            els.btnSpell.textContent = spellLabel();
        }

        els.btnSpell.addEventListener('click', () => {
            if (!spellAvailable || phase !== 'ACTION' || busy) return;
            setTargeting(!isTargeting);
            if (isTargeting) showToast("Target an ally to Double, or an enemy to Destroy.");
        });

        document.addEventListener('keydown', (e) => {
            if (e.key === 'Escape' && isTargeting) setTargeting(false);
        });

        async function handleCardClick(isOpponent, index) {
            if (!isTargeting || !spellAvailable || phase !== 'ACTION' || busy) return;

            const targetArmy = isOpponent ? opponentArmy : playerArmy;
            const targetStack = targetArmy[index];
            if (!targetStack || targetStack.qty === 0) return;

            spellAvailable = false;
            busy = true;
            setTargeting(false);
            updateUI();

            if (isOpponent) {
                // Implosion
                targetStack.qty = 0;
                showToast("Implosion Cast! Enemy card destroyed.");
                await playSpell(`opp-card-${index}`, 'anim-void');
            } else {
                // Animate Dead
                targetStack.qty *= 2;
                targetStack.empowered = true;
                showToast("Animate Dead Cast! Your card's points are doubled and it is upgraded.");
                await empowerWithAnimation(`player-card-${index}`);
            }

            busy = false;
            updateUI();
        }

        // Plays a spell animation and resolves when it finishes
        function playSpell(elementId, animClass) {
            return new Promise(resolve => {
                const el = document.getElementById(elementId);
                if (!el) {
                    resolve();
                    return;
                }
                let done = false;
                const finish = () => {
                    if (done) return;
                    done = true;
                    el.removeEventListener('animationend', onEnd);
                    // A voided card stays collapsed until the board re-renders without it
                    if (animClass !== 'anim-void') el.classList.remove(animClass);
                    resolve();
                };
                const onEnd = (e) => { if (e.target === el) finish(); };

                el.classList.remove('animate-land');
                void el.offsetWidth; // Restart animation cleanly
                el.classList.add(animClass);
                el.addEventListener('animationend', onEnd);
                setTimeout(finish, 1500); // Safety net if animationend never fires
            });
        }

        // Animate Dead: the new quantity and empowered art appear at the brightest moment of the flame
        async function empowerWithAnimation(elementId) {
            const spell = playSpell(elementId, 'anim-spectral');
            await wait(400);
            updateUI();
            await spell;
        }
```

```
1445
1446        // --- RESOLVE & AI ---
1447        els.btnResolve.addEventListener('click', async () => {
1448            if (phase !== 'ACTION' || busy) return;
1449            phase = 'RESOLVED';
1450            setTargeting(false);
1451
1452            // 1. Lift Fog of War (opponent cards flip face up)
1453            fogOfWar = false;
1454            updateUI();
1455
1456            // 2. Delay slightly for dramatic effect, then AI casts spell
1457            await wait(800);
1458            await aiTurn();
1459            updateUI();
1460
1461            // 3. Evaluate Round / Game Over
1462            await wait(1500);
1463            if (currentRound >= TOTAL_ROUNDS) {
1464                finalizeGame();
1465                return;
1466            }
1467            currentRound++;
1468            phase = 'BETTING';
1469            fogOfWar = true; // Fog returns next round
1470            updateUI();
1471            showToast(`Round ${currentRound} begins. Your bet is locked in: draw your next five cards.`);
1472        });
1473
1474        // The AI usually plays smart (AI_SMART_CHANCE): it destroys your best card if that is worth more
1475        // than doubling its own best card, otherwise it doubles its own. The rest of the time it casts at
1476        // random: a coin flip between Animate Dead on one of its cards and Implosion on one of yours.
1477        async function aiTurn() {
1478            const livePiles = army => army.map((stack, i) => (stack && stack.qty > 0 ? i : -1)).filter(i => i >= 0);
1479            const pick = list => list[Math.floor(Math.random() * list.length)];
1480            const own = livePiles(opponentArmy);
1481            const yours = livePiles(playerArmy);
1482            if (own.length === 0 && yours.length === 0) return;
1483
1484            let castImplosion, index;
1485            if (Math.random() < AI_SMART_CHANCE) {
1486                const strongest = (army, piles) => piles.reduce((best, i) => (stackPoints(army[i]) > stackPoints(army[bes
     t]) ? i : best), piles[0]);
1487                const yourBest = yours.length ? strongest(playerArmy, yours) : -1;
1488                const ownBest = own.length ? strongest(opponentArmy, own) : -1;
1489                const destroyValue = yourBest === -1 ? 0 : stackPoints(playerArmy[yourBest]);
1490                const doubleValue = ownBest === -1 ? 0 : stackPoints(opponentArmy[ownBest]); // Doubling adds exactly its
     current value
1491                castImplosion = destroyValue > doubleValue;
1492                index = castImplosion ? yourBest : ownBest;
1493            } else {
1494                castImplosion = yours.length > 0 && (own.length === 0 || Math.random() < 0.5);
1495                index = castImplosion ? pick(yours) : pick(own);
1496            }
1497
1498            if (castImplosion) {
1499                playerArmy[index].qty = 0;
1500                showToast("Opponent casts Implosion on your forces!", 2000);
1501                await playSpell(`player-card-${index}`, 'anim-void');
1502            } else {
1503                opponentArmy[index].qty *= 2;
1504                opponentArmy[index].empowered = true;
1505                showToast("Opponent casts Animate Dead! Their card's points are doubled and it is upgraded.", 2000);
1506                await empowerWithAnimation(`opp-card-${index}`);
1507            }
1508        }
1509
1510        // A win returns your bet plus winnings of bet x 0.20 x (your points / 100), rounded down to whole gold.
1511        // Tuned with the AI's smart chance so even the strongest strategy found returns about 96% (RTP)
1512        const WIN_RATE_PERCENT = 20; // 0.20 of the bet for every 100 points in your army
1513        function winWinnings(bet, playerPts) {
1514            return Math.floor(bet * playerPts * WIN_RATE_PERCENT / 10000);
1515        }
1516
1517        // Crypt Fortune: after every win a luck roll may multiply the winnings (the bet itself comes back once).
1518        // It is rolled after the result with the bet already locked, so no strategy can change the odds.
1519        // Budgeted so even the strongest strategy found stays at about 97.4% RTP (limit 97.5%)
1520        const FORTUNE_TIERS = [
1521            { name: 'MEGA WIN', key: 'mega', multiplier: 50, chance: 1 / 20000 },
1522            { name: 'BIG WIN', key: 'big', multiplier: 10, chance: 1 / 1000 },
1523            { name: 'Lucky Win', key: 'lucky', multiplier: 2, chance: 1 / 50 },
1524        ];
1525        const FORTUNE_FACES = [1, 2, 10, 50];
1526
1527        function rollFortune() {
1528            let roll = Math.random();
```

```
            for (const tier of FORTUNE_TIERS) {
                if (roll < tier.chance) return tier;
                roll -= tier.chance;
            }
            return null; // A normal win
        }

        // Spins the reel, lands on the rolled multiplier and shows what it paid
        async function showFortune(tier, bet, winnings) {
            const multiplier = tier ? tier.multiplier : 1;
            els.fortune.className = 'fortune';
            els.fortuneHeadline.textContent = '';
            els.fortuneResult.textContent = '';
            els.fortune.hidden = false;
            if (!REDUCE_MOTION) {
                for (let step = 0, delay = 40; step < 14; step++, delay *= 1.15) {
                    els.fortuneReel.textContent = `×${FORTUNE_FACES[step % FORTUNE_FACES.length]}`;
                    await wait(delay);
                }
            }
            els.fortuneReel.textContent = `×${multiplier}`;
            if (tier) {
                els.fortune.classList.add(`tier-${tier.key}`);
                els.fortuneHeadline.textContent = `${tier.name}!`;
                els.fortuneResult.textContent = `Your ${winnings} gold winnings ×${multiplier} = ${winnings * multiplier} gold. You collect ${bet + winnings * multiplier} gold.`;
            } else {
                els.fortuneHeadline.textContent = 'Victory';
                els.fortuneResult.textContent = `No fortune this time. You collect ${bet + winnings} gold.`;
            }
            await wait(tier ? 3200 : 1500);
            els.fortune.hidden = true;
        }

        async function finalizeGame() {
            const pStrength = getArmyStrength(playerArmy);
            const oStrength = getArmyStrength(opponentArmy);

            if (pStrength > oStrength) {
                const bet = totalPot;
                const winnings = winWinnings(bet, pStrength);
                const fortune = rollFortune();
                const multiplier = fortune ? fortune.multiplier : 1;
                const collected = bet + winnings * multiplier;
                // Credit the gold before the reveal so closing the page mid-spin can't lose a win
                gold += collected;
                saveGold();
                await showFortune(fortune, bet, winnings);
                showToast(fortune
                    ? `${fortune.name}! Winnings ×${multiplier}: you collect ${collected} gold.`
                    : `VICTORY with ${pStrength} pts! Your ${bet} gold bet comes back with ${winnings} gold winnings.`, 5000);
            } else if (oStrength > pStrength) {
                showToast("DEFEAT. The Necropolis claims your soul and your gold.", 5000);
            } else {
                gold += totalPot;
                showToast("DRAW. The dead rest easy. Your bet is returned.", 5000);
            }
            saveGold();
            updateUI();

            // Reset Game after 5 seconds
            setTimeout(initGame, 5000);
        }

        // --- RENDER ---
        function updateUI() {
            els.roundDisp.textContent = `${currentRound} / ${TOTAL_ROUNDS}`;
            els.potDisp.textContent = totalPot;
            els.goldDisp.textContent = gold;
            els.deckDisp.textContent = deck.length;
            els.deckPile.classList.toggle('empty', deck.length === 0);

            // Buttons state
            els.btnDraw.disabled = (phase !== 'BETTING' || busy);
            els.btnSpell.disabled = (!spellAvailable || phase !== 'ACTION' || busy);
            els.btnSpell.textContent = spellLabel();
            els.btnResolve.disabled = (phase !== 'ACTION' || busy);
            els.betInput.disabled = (phase !== 'BETTING' || busy || currentRound > 1);

            // Strengths
            els.playerStrength.textContent = getArmyStrength(playerArmy);
            els.oppStrength.textContent = fogOfWar ? '???' : getArmyStrength(opponentArmy);

            // Render Boards
            renderArmy(playerArmy, els.playerSlots, false);
```

```
            renderArmy(opponentArmy, els.oppSlots, true);
        }

        // Updates the board in place so flips, spell effects and quantity changes animate
        // instead of every card being rebuilt (and re-dealt) on each update
        function renderArmy(army, container, isOpponent) {
            if (container.children.length !== SLOT_COUNT) {
                container.innerHTML = '';
                // Always render 5 physical slots
                for (let i = 0; i < SLOT_COUNT; i++) {
                    const slot = document.createElement('div');
                    slot.className = 'card-slot';
                    slot.style.zIndex = SLOT_COUNT - i; // Left cards sit over right ones, so a pile's x2 badge is never c
overed
                    slot.addEventListener('click', () => handleCardClick(isOpponent, i));
                    slot.addEventListener('keydown', (e) => {
                        if ((e.key === 'Enter' || e.key === ' ') && e.target.classList.contains('card-wrapper')) {
                            e.preventDefault();
                            handleCardClick(isOpponent, i);
                        }
                    });
                    container.appendChild(slot);
                }
            }

            for (let i = 0; i < SLOT_COUNT; i++) {
                const slot = container.children[i];
                const stack = army[i];
                const alive = Boolean(stack && stack.qty > 0);
                const key = alive ? stack.id : '';

                if (slot.dataset.key !== key) {
                    slot.dataset.key = key;
                    // A new card settles in; empty slots show a dashed outline
                    slot.innerHTML = alive
                        ? buildCardHTML(stack, i, isOpponent)
                        : '<div class="slot-empty"></div>';
                }
                if (!alive) continue;

                // Apply Fog of War ONLY to opponent
                const hidden = isOpponent && fogOfWar;
                slot.querySelector('.card').classList.toggle('face-down', hidden);
                slot.querySelector('.card-wrapper').setAttribute('aria-label', hidden
                    ? 'Hidden enemy card'
                    : `${isOpponent ? 'Enemy' : 'Your'} ${stack.empowered ? 'upgraded ' : ''}level ${stack.level} ${stack.
name}, ${stackPoints(stack)} points`);

                const art = slot.querySelector('.card-art');
                if (art && art.getAttribute('src') !== artSrc(stack, stack.empowered)) {
                    art.setAttribute('src', artSrc(stack, stack.empowered));
                }

                // Points pulse whenever they change (a matching card joins, or Animate Dead)
                const points = slot.querySelector('.card-points');
                const pointsText = String(stackPoints(stack));
                const value = points.querySelector('.card-points-value');
                if (value.textContent !== pointsText) {
                    value.textContent = pointsText;
                    pulse(points);
                }

                // Matching cards: a pile behind the card and an x2, x3 badge (single cards show neither)
                const wrapper = slot.querySelector('.card-wrapper');
                wrapper.classList.toggle('stack-2', stack.qty === 2);
                wrapper.classList.toggle('stack-3', stack.qty >= 3);
                const count = slot.querySelector('.card-count');
                const countText = `x${stack.qty}`;
                count.hidden = stack.qty < 2;
                if (count.textContent !== countText) {
                    count.textContent = countText;
                    if (!count.hidden) pulse(count);
                }
            }
        }

        // Replays the little bump animation on a badge whose number just changed
        function pulse(el) {
            el.classList.remove('badge-bump');
            void el.offsetWidth;
            el.classList.add('badge-bump');
        }

        // The face of a card: creature art, name and the points it gives
        function cardFrontHTML(stack) {
            // Creature art fills the card; units without art keep the skull sigil
```

```
            const artHtml = stack.art
                ? `<img class="card-art" src="${artSrc(stack, stack.empowered)}" alt="" draggable="false">`
                : '';

            return `
                <div class="card-face card-front ${stack.art ? 'has-art' : ''}" style="--lv: ${stack.color};">
                    ${artHtml}
                    <div class="card-name">${stack.name}</div>
                    <div class="card-sigil">${stack.art ? '' : '☠'}</div>
                    <div class="card-foot">
                        <span class="card-points"><span class="card-points-value">${stackPoints(stack)}</span><small>pts</small></span>
                    </div>
                </div>
            `;
        }

        function buildCardHTML(stack, index, isOpponent) {
            const isFacedown = isOpponent && fogOfWar;
            const prefix = isOpponent ? 'opp' : 'player';

            // Random positioning applied to the wrapper inside the slot
            const transformStyle = `transform: translate(${stack.tx}px, ${stack.ty}px) rotate(${stack.rot}deg);`;

            return `
                <div class="card-wrapper" style="${transformStyle}" tabindex="0" role="button">
                    <div id="${prefix}-card-${index}" class="card-fx animate-land">
                        <div class="card ${isFacedown ? 'face-down' : ''}">
                            ${cardFrontHTML(stack)}
                            <div class="card-face card-back"></div>
                        </div>
                        <div class="card-count" hidden></div>
                    </div>
                </div>
            `;
        }

        // --- RUN ---
        // Keeps a hand in progress when the page is hot-reloaded in the Claude artifact viewer (no-op elsewhere)
        window.claude?.hot?.snapshot?.(() => (phase === 'RESOLVED' || busy) ? null : {
            deck, playerArmy, opponentArmy, currentRound, totalPot, phase, fogOfWar, spellAvailable
        });

        function start(saved) {
            if (saved && Array.isArray(saved.deck)) {
                ({ deck, playerArmy, opponentArmy, currentRound, totalPot, phase, fogOfWar, spellAvailable } = saved);
                // Cards saved by an older version of the page may lack newer fields such as art
                [...deck, ...playerArmy, ...opponentArmy].forEach(card => {
                    const unit = UNIT_DATA.find(u => u.name === card.name);
                    if (unit) Object.assign(card, { level: unit.level, pts: unit.pts, color: unit.color, art: unit.art });
                });
                setTargeting(false);
                updateUI();
            } else {
                initGame();
            }
        }

        window.claude?.hot?.ready ? window.claude.hot.ready(start) : start(window.claude?.hot?.data ?? {});
    </script>
</body>
</html>
```

multiplayer/server.js

Two-player game server / ორმოთამაშიანი თამაშის სერვერი · 483 lines · SHA-256 b7d85146abd4000903dbe06d5ba589564e05f6a4aae4f6f2de9143d1d9caa95c

```
// Necropolis Casino: two-player server.
// Runs the real game (deck, deals, spells, scores, payouts) and sends each player only what
// they are allowed to see, so the fog of war can't be bypassed from the browser.
// No packages needed. Start it with:   node server.js        (then open http://localhost:8080)
'use strict';

const http = require('http');
const fs = require('fs');
const path = require('path');
const os = require('os');
const crypto = require('crypto');
const { spawn } = require('child_process');

const PORT = Number(process.env.PORT || 8080);
const CLIENT_FILE = path.join(__dirname, 'client.html');
const SEAT_RECLAIM_MS = 60 * 1000; // A seat whose player has been gone this long can be taken by someone new

// --- Game rules (the same as the single-player game) ---
const UNITS = [
    { name: 'Skeleton', spawn: 20, power: 1, count: 20 },
    { name: 'Zombie', spawn: 12, power: 2, count: 12 },
    { name: 'Wight', spawn: 9, power: 3, count: 9 },
    { name: 'Vampire', spawn: 8, power: 4, count: 8 },
    { name: 'Lich', spawn: 6, power: 7, count: 6 },
    { name: 'Dark Knight', spawn: 3, power: 17, count: 3 },
    { name: 'Bone Dragon', spawn: 1, power: 60, count: 1 },
].map(unit => ({ ...unit, pts: unit.spawn * unit.power }));
const PTS = Object.fromEntries(UNITS.map(unit => [unit.name, unit.pts]));

const SLOT_COUNT = 5;
const CARDS_PER_ROUND = 5;   // Each army draws this many cards every round
const TOTAL_ROUNDS = 3;
const PARTNER_RANGE = 10;    // A dealt pair differs by at most this many points when the crypt allows
const WIN_RATE_PERCENT = 20; // A win returns the bet plus bet x 0.20 x (your points / 100)
const FORTUNE_TIERS = [      // Crypt Fortune: after a win, a luck roll may multiply the winnings
    { key: 'mega', multiplier: 50, chance: 1 / 20000 },
    { key: 'big', multiplier: 10, chance: 1 / 1000 },
    { key: 'lucky', multiplier: 2, chance: 1 / 50 },
];

const random = () => crypto.randomInt(0, 2 ** 47) / 2 ** 47; // Uniform in [0, 1) from a secure source

// --- Table state ---
// phase: lobby -> betting -> spells -> reveal -> spells ... -> finished -> betting (play again)
const table = { seats: [null, null], phase: 'lobby', round: 0, deck: [], gameId: 0, result: null };
let nextPileId = 1;

function newSeat(name) {
    return {
        token: crypto.randomUUID(), name, army: [], bet: 0, ready: false, spell: null,
        outbox: [], seq: 0, waiters: new Set(), online: false, awayTimer: null, goneSince: Date.now(),
    };
}

function buildDeck() {
    const deck = [];
    for (const unit of UNITS) for (let i = 0; i < unit.count; i++) deck.push({ name: unit.name, pts: unit.pts });
    for (let i = deck.length - 1; i > 0; i--) {
        const j = crypto.randomInt(i + 1);
        [deck[i], deck[j]] = [deck[j], deck[i]];
    }
    return deck;
}

const strength = army => army.reduce((sum, pile) => sum + (pile && pile.qty > 0 ? PTS[pile.name] * pile.qty : 0), 0);

function freeSlot(army) {
    for (let i = 0; i < SLOT_COUNT; i++) if (!army[i] || army[i].qty === 0) return i;
    return -1;
}
const fits = (army, card) => army.some(pile => pile && pile.qty > 0 && pile.name === card.name) || freeSlot(army) !== -1;

// Puts a card onto its matching pile or into the first free slot
function place(army, card) {
    const index = army.findIndex(pile => pile && pile.qty > 0 && pile.name === card.name);
    if (index !== -1) {
        army[index].qty += 1;
        return { index, joins: true, card };
    }
    const free = freeSlot(army);
    const pile = {
```

```
        id: `p${nextPileId++}`, name: card.name, qty: 1, empowered: false, revealed: false,
        rot: random() * 12 - 6, tx: random() * 10 - 5, ty: random() * 10 - 5, // Tabletop scatter
    };
    army[free] = pile;
    return { index: free, joins: false, card, pile: { id: pile.id, name: pile.name, rot: pile.rot, tx: pile.tx, ty: pile.t
y } };
}

// The leading side draws random cards until one fits (a card with nowhere to go is discarded)
function drawLead(army) {
    const discards = [];
    while (table.deck.length > 0) {
        const card = table.deck.pop();
        if (fits(army, card)) return { discards, landing: place(army, card) };
        discards.push(card);
    }
    return { discards, landing: null };
}

// The other side gets a random card that fits, worth within PARTNER_RANGE of the leader's card
function drawPartner(army, leadCard) {
    const fitting = [];
    table.deck.forEach((card, i) => { if (fits(army, card)) fitting.push(i); });
    if (fitting.length === 0) return null;
    const gap = i => Math.abs(table.deck[i].pts - leadCard.pts);
    let pool = fitting.filter(i => gap(i) <= PARTNER_RANGE);
    if (pool.length === 0) {
        const closest = Math.min(...fitting.map(gap));
        pool = fitting.filter(i => gap(i) === closest);
    }
    const [card] = table.deck.splice(pool[crypto.randomInt(pool.length)], 1);
    return place(army, card);
}

// Deals one round in matched pairs and returns what happened, for the animations
function dealRound() {
    const pairs = [];
    for (let i = 0; i < CARDS_PER_ROUND; i++) {
        const leader = crypto.randomInt(2);
        const lead = drawLead(table.seats[leader].army);
        if (!lead.landing) {
            if (lead.discards.length) pairs.push({ steps: [{ seat: leader, discards: lead.discards, landing: null }] });
            break; // The crypt is empty
        }
        const steps = [{ seat: leader, discards: lead.discards, landing: lead.landing }];
        const partner = drawPartner(table.seats[1 - leader].army, lead.landing.card);
        if (partner) steps.push({ seat: 1 - leader, discards: [], landing: partner });
        pairs.push({ steps });
    }
    return pairs;
}

function rollFortune() {
    let roll = random();
    for (const tier of FORTUNE_TIERS) {
        if (roll < tier.chance) return tier;
        roll -= tier.chance;
    }
    return null;
}

// --- What each player may see ---
function pileView(pile, isOwner) {
    if (!pile || pile.qty <= 0) return null;
    const view = { id: pile.id, qty: pile.qty, rot: pile.rot, tx: pile.tx, ty: pile.ty };
    // The owner always knows the card; the opponent only once it has been revealed at a round's end
    if (isOwner || pile.revealed) return { ...view, name: pile.name, empowered: pile.empowered, known: true };
    return { ...view, known: false };
}

function stateFor(i) {
    const me = table.seats[i];
    const opp = table.seats[1 - i];
    return {
        type: 'state',
        gameId: table.gameId,
        phase: table.phase,
        round: table.round,
        deckCount: table.deck.length,
        fog: !(table.phase === 'reveal' || table.phase === 'finished'),
        you: { seat: i, name: me.name, bet: me.bet, ready: me.ready, spell: me.spell, army: armyView(me.army, true) },
        opp: opp ? {
            name: opp.name, connected: opp.online, betPlaced: opp.bet > 0,
            ready: opp.ready, spellChosen: Boolean(opp.spell), army: armyView(opp.army, false),
        } : null,
        result: table.phase === 'finished' && table.result ? table.result[i] : null,
```

```
    };
}
const armyView = (army, isOwner) => Array.from({ length: SLOT_COUNT }, (_, i) => pileView(army[i], isOwner));

function dealEventFor(i, pairs) {
    return {
        type: 'deal',
        round: table.round,
        pairs: pairs.map(pair => ({
            steps: pair.steps.map(step => {
                const mine = step.seat === i;
                return {
                    side: mine ? 'you' : 'opp',
                    discards: step.discards.map(card => (mine ? { name: card.name } : {})),
                    landing: step.landing && {
                        index: step.landing.index,
                        joins: step.landing.joins,
                        card: mine ? { name: step.landing.card.name } : null,
                        pile: step.landing.pile && (mine
                            ? { ...step.landing.pile, qty: 1, known: true }
                            : { id: step.landing.pile.id, rot: step.landing.pile.rot, tx: step.landing.pile.tx, ty: step.l
anding.pile.ty, qty: 1, known: false }),
                    },
                };
            }),
        })),
    };
}

// --- Sending to players (long polling) ---
// Each browser keeps asking "anything new?" and the server answers as soon as something happens.
// Plain requests and answers get through every tunnel and proxy, unlike a stream held open.
const POLL_WAIT_MS = 25 * 1000; // How long a question waits for news before the answer is "nothing new"
const AWAY_MS = 8 * 1000;       // A player whose browser stops asking for this long is shown as away
const OUTBOX_SIZE = 200;        // Recent messages kept per player, to resend after a short drop

function send(i, message) {
    const seat = table.seats[i];
    if (!seat) return;
    seat.outbox.push({ seq: ++seat.seq, message });
    if (seat.outbox.length > OUTBOX_SIZE) seat.outbox.shift();
    for (const waiter of [...seat.waiters]) answerPoll(seat, waiter);
}
function answerPoll(seat, waiter) {
    seat.waiters.delete(waiter);
    clearTimeout(waiter.timer);
    const messages = seat.outbox.filter(entry => entry.seq > waiter.after).map(entry => entry.message);
    reply(waiter.res, 200, { seq: seat.seq, messages });
}
function setOnline(i, seat, online) {
    if (table.seats[i] !== seat || seat.online === online || (!online && seat.waiters.size > 0)) return;
    seat.online = online;
    seat.goneSince = online ? null : Date.now();
    if (table.seats[1 - i]) send(1 - i, stateFor(1 - i)); // The opponent sees you come and go
}
function broadcastState() {
    for (const i of [0, 1]) if (table.seats[i]) send(i, stateFor(i));
}
function notify(i, text) {
    send(i, { type: 'info', text });
}

// --- Game flow ---
function newGame() {
    table.gameId += 1;
    table.phase = 'betting';
    table.round = 0;
    table.deck = buildDeck();
    table.result = null;
    for (const seat of table.seats) Object.assign(seat, { army: [], bet: 0, ready: false, spell: null });
    broadcastState();
}

function startRound(round) {
    table.round = round;
    for (const seat of table.seats) Object.assign(seat, { ready: false, spell: null });
    const pairs = dealRound();
    for (const i of [0, 1]) send(i, dealEventFor(i, pairs));
    table.phase = 'spells';
    broadcastState();
}

// Both spells take effect together: doubles first, then implosions,
// so a card that is doubled and destroyed in the same round is destroyed
function resolveRound() {
    const casts = [0, 1].map(i => table.seats[i].spell && { caster: i, ...table.seats[i].spell }).filter(Boolean);
```

```
    const ordered = [...casts.filter(c => c.type === 'double'), ...casts.filter(c => c.type === 'implode')];

    // The end of a round reveals both armies
    for (const seat of table.seats) for (const pile of seat.army) if (pile && pile.qty > 0) pile.revealed = true;
    const before = [0, 1].map(i => armyView(table.seats[i].army, true));

    for (const cast of ordered) {
        if (cast.type === 'double') {
            const pile = table.seats[cast.caster].army[cast.index];
            if (pile && pile.qty > 0) { pile.qty *= 2; pile.empowered = true; }
        } else {
            const pile = table.seats[1 - cast.caster].army[cast.index];
            if (pile && pile.qty > 0) pile.qty = 0;
        }
    }

    for (const i of [0, 1]) {
        send(i, {
            type: 'resolve',
            round: table.round,
            before: { you: before[i], opp: before[1 - i] },
            spells: ordered.map(cast => ({
                caster: cast.caster === i ? 'you' : 'opp',
                type: cast.type,
                side: (cast.type === 'double') === (cast.caster === i) ? 'you' : 'opp',
                index: cast.index,
            })),
        });
    }

    for (const seat of table.seats) Object.assign(seat, { ready: false, spell: null });
    if (table.round < TOTAL_ROUNDS) {
        table.phase = 'reveal';
        broadcastState();
    } else {
        finishGame();
    }
}

function finishGame() {
    const points = table.seats.map(seat => strength(seat.army));
    table.result = [0, 1].map(i => {
        const mine = points[i], theirs = points[1 - i], bet = table.seats[i].bet;
        if (mine > theirs) {
            const winnings = Math.floor(bet * mine * WIN_RATE_PERCENT / 10000);
            const fortune = rollFortune();
            return { outcome: 'win', you: mine, opp: theirs, bet, winnings, fortune: fortune ? fortune.key : null,
                     collected: bet + winnings * (fortune ? fortune.multiplier : 1) };
        }
        if (mine < theirs) return { outcome: 'loss', you: mine, opp: theirs, bet, winnings: 0, fortune: null, collected: 0 };
        return { outcome: 'draw', you: mine, opp: theirs, bet, winnings: 0, fortune: null, collected: bet };
    });
    table.phase = 'finished';
    for (const i of [0, 1]) send(i, { type: 'gameover', gameId: table.gameId, ...table.result[i] });
    broadcastState();
}

// --- Player actions ---
function act(i, msg) {
    const me = table.seats[i];
    const opp = table.seats[1 - i];
    if (!opp) return 'Waiting for a second player to join.';
    const bothReady = () => table.seats.every(seat => seat.ready);

    switch (msg.type) {
        case 'bet': {
            if (table.phase !== 'betting' || me.ready) return 'Bets are closed right now.';
            const amount = Number(msg.amount);
            if (!Number.isInteger(amount) || amount < 1 || amount > 1e12) return 'Bet a whole number of gold, at least 1.';
            me.bet = amount;
            me.ready = true;
            notify(1 - i, `${me.name} placed a bet.`);
            if (bothReady()) startRound(1); else broadcastState();
            return null;
        }
        case 'spell': {
            if (table.phase !== 'spells' || me.ready) return 'You cannot cast a spell right now.';
            const spell = msg.spell;
            if (spell === null) { me.spell = null; broadcastState(); return null; }
            const army = spell && spell.type === 'double' ? me.army : spell && spell.type === 'implode' ? opp.army : null;
            if (!army || !Number.isInteger(spell.index) || spell.index < 0 || spell.index >= SLOT_COUNT) return 'Unknown spell.';
            const target = army[spell.index];
            if (!target || target.qty <= 0) return 'That card is not on the table.';
```

```
            me.spell = { type: spell.type, index: spell.index };
            broadcastState();
            return null;
        }
        case 'ready': {
            if (table.phase !== 'spells' || me.ready) return 'You are already ready.';
            me.ready = true;
            if (bothReady()) resolveRound(); else broadcastState();
            return null;
        }
        case 'continue': {
            if (table.phase !== 'reveal' || me.ready) return 'Not yet.';
            me.ready = true;
            if (bothReady()) startRound(table.round + 1); else broadcastState();
            return null;
        }
        case 'again': {
            if (table.phase !== 'finished' || me.ready) return 'Not yet.';
            me.ready = true;
            if (bothReady()) newGame(); else broadcastState();
            return null;
        }
        default:
            return 'Unknown action.';
    }
}

// Seats a player: same token rejoins; otherwise a free seat, or a seat abandoned for a while
function join(name, token) {
    const existing = table.seats.findIndex(seat => seat && seat.token === token);
    if (existing !== -1) return { seat: existing, token };

    let index = table.seats.findIndex(seat => !seat);
    if (index === -1) {
        index = table.seats.findIndex(seat => !seat.online && Date.now() - seat.goneSince > SEAT_RECLAIM_MS);
        if (index === -1) return { error: 'The table is full: two players are already playing.' };
        const other = 1 - index;
        notify(other, `${table.seats[index].name} left the table. ${name} takes the empty seat; a new game begins.`);
    }
    table.seats[index] = newSeat(name);
    if (table.seats[0] && table.seats[1]) newGame();
    else { table.phase = 'lobby'; broadcastState(); }
    return { seat: index, token: table.seats[index].token };
}

// --- HTTP ---
function readJson(req) {
    return new Promise((resolve, reject) => {
        let body = '';
        req.on('data', chunk => {
            body += chunk;
            if (body.length > 10000) { reject(new Error('Request too large')); req.destroy(); }
        });
        req.on('end', () => { try { resolve(JSON.parse(body || '{}')); } catch (e) { reject(e); } });
    });
}
function reply(res, status, data) {
    res.writeHead(status, { 'Content-Type': 'application/json', 'Cache-Control': 'no-store' });
    res.end(JSON.stringify(data));
}
const seatOf = token => table.seats.findIndex(seat => seat && token && seat.token === token);

const server = http.createServer(async (req, res) => {
    const url = new URL(req.url, 'http://localhost');
    try {
        if (req.method === 'GET' && (url.pathname === '/' || url.pathname === '/index.html')) {
            res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-Control': 'no-store' });
            fs.createReadStream(CLIENT_FILE).pipe(res);
            return;
        }
        if (req.method === 'GET' && url.pathname === '/poll') {
            const i = seatOf(url.searchParams.get('token'));
            if (i === -1) return reply(res, 401, { error: 'Unknown player. Join the table again.' });
            const seat = table.seats[i];
            const waiter = { res, after: Number(url.searchParams.get('after') ?? NaN), timer: null };
            clearTimeout(seat.awayTimer);
            seat.waiters.add(waiter);
            res.on('close', () => {
                clearTimeout(waiter.timer);
                seat.waiters.delete(waiter);
                if (table.seats[i] === seat && seat.waiters.size === 0) {
                    clearTimeout(seat.awayTimer);
                    seat.awayTimer = setTimeout(() => setOnline(i, seat, false), AWAY_MS);
                }
            });
            setOnline(i, seat, true);
```

```
            const oldest = seat.outbox.length ? seat.outbox[0].seq : seat.seq + 1;
            if (!Number.isInteger(waiter.after) || waiter.after < oldest - 1 || waiter.after > seat.seq) {
                // A freshly opened page (or one that missed too much) starts from the table as it is now
                seat.waiters.delete(waiter);
                return reply(res, 200, { seq: seat.seq, messages: [stateFor(i)] });
            }
            if (waiter.after < seat.seq) return answerPoll(seat, waiter);
            waiter.timer = setTimeout(() => answerPoll(seat, waiter), POLL_WAIT_MS);
            return;
        }
        if (req.method === 'POST' && url.pathname === '/join') {
            const body = await readJson(req);
            const name = String(body.name || '').trim().slice(0, 20) || 'Necromancer';
            const result = join(name, body.token);
            return reply(res, result.error ? 409 : 200, result);
        }
        if (req.method === 'POST' && url.pathname === '/action') {
            const body = await readJson(req);
            const i = seatOf(body.token);
            if (i === -1) return reply(res, 401, { error: 'Unknown player. Join the table again.' });
            const error = act(i, body);
            return reply(res, error ? 400 : 200, error ? { error } : { ok: true });
        }
        if (url.pathname === '/favicon.ico') { res.writeHead(204); res.end(); return; }
        reply(res, 404, { error: 'Not found' });
    } catch (e) {
        console.error('Request failed:', e);
        reply(res, 400, { error: 'Bad request' });
    }
});

server.listen(PORT, () => {
    console.log(`Necropolis Casino is running.`);
    console.log(`  On this PC:            http://localhost:${PORT}`);
    for (const nets of Object.values(os.networkInterfaces())) {
        for (const net of nets || []) {
            if (net.family === 'IPv4' && !net.internal) console.log(`  On your Wi-Fi/network: http://${net.address}:${POR
T}`);
        }
    }
    console.log('Keep this window open while you play. Press Ctrl+C to stop the server.');
    if (!process.env.NO_BROWSER) openBrowser(`http://localhost:${PORT}`);
});
server.on('error', (e) => {
    if (e.code === 'EADDRINUSE') {
        console.error(`Port ${PORT} is already in use, so the server is probably already running in another window.`);
        console.error(`Use that window and open http://localhost:${PORT}, or close it and start this again.`);
    } else {
        console.error('The server could not start:', e.message);
    }
    process.exitCode = 1;
});

// Opens the game for the host on Windows and macOS (set NO_BROWSER=1 to skip)
function openBrowser(url) {
    const command = process.platform === 'win32' ? ['cmd', ['/c', 'start', '""', url]]
        : process.platform === 'darwin' ? ['open', [url]] : null;
    if (!command) return;
    try {
        const child = spawn(command[0], command[1], { stdio: 'ignore', detached: true, windowsVerbatimArguments: true });
        child.on('error', () => {});
        child.unref();
    } catch (e) { /* The address is printed above, so it can be opened by hand */ }
}
```

multiplayer/client.html

Two-player game page / ორმოთამაშიანი თამაშის გვერდი · 1912 lines · SHA-256 8240b8799215005fd19e5ba28b2f4fae9deabeff9dcb9bb363b63afc4d7352e9

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover">
    <title>Necropolis Casino Duel</title>
    <link href="https://fonts.googleapis.com/css2?family=Exo+2:wght@400;500;600;700;800&family=Orbitron:wght@600;700;900&display=swap" rel="stylesheet">
    <style>
        /* Neon Necropolis: bright neon accents on calm, solid dark surfaces.
           Layout: enemy rank on top, wager belt across the middle, your rank below;
           on phones the action buttons sit in a bar at the bottom, within thumb reach. */
        :root {
            color-scheme: dark;
            --bg: #070812;
            --surface: #0f1124;
            --surface-hi: #151833;
            --fg: #e7eaff;
            --muted: #a0a6cc;
            --faint: #646a96;
            --line: rgba(140, 150, 255, 0.16);
            --line-strong: rgba(140, 150, 255, 0.32);
            --card-ink: #0a0b18;
            /* Neon accents, as "r g b" so they can take any alpha */
            --you-rgb: 46 230 255;      /* Your side: cyan */
            --foe-rgb: 255 61 154;      /* The opponent's side: magenta */
            --life-rgb: 77 255 136;     /* Animate Dead: green */
            --void-rgb: 180 107 255;    /* Implosion and the crypt: violet */
            --gold-rgb: 255 216 77;     /* Bets and fortune: gold */
            --danger-rgb: 255 77 109;   /* Resolve and losses: red */
            --you: rgb(var(--you-rgb));
            --foe: rgb(var(--foe-rgb));
            --ally: rgb(var(--life-rgb));
            --enemy: rgb(var(--void-rgb));
            --void: rgb(var(--void-rgb));
            --gold: rgb(var(--gold-rgb));
            --blood: rgb(var(--danger-rgb));
            --font-display: 'Orbitron', 'Exo 2', system-ui, sans-serif;
            --font-ui: 'Exo 2', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
            --font-num: 'Exo 2', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif; /* Plain digits read fastest */
            --radius: 14px;
            --card-radius: 10px;
            --bar-h: 0px;
            /* Card width: five cards plus gaps and gutters always fit the screen */
            --cw: min(130px, calc((100vw - 130px) / 5));
        }

        html { height: 100%; -webkit-text-size-adjust: 100%; }

        body {
            min-height: 100%;
            margin: 0;
            display: flex;
            flex-direction: column;
            font-family: var(--font-ui);
            color: var(--fg);
            background-color: var(--bg);
            /* A faint neon grid under soft violet and cyan light */
            background-image:
                radial-gradient(ellipse 90% 45% at 50% 0%, rgb(var(--void-rgb) / 0.13), transparent 70%),
                radial-gradient(ellipse 90% 45% at 50% 100%, rgb(var(--you-rgb) / 0.09), transparent 70%),
                linear-gradient(rgb(140 150 255 / 0.035) 1px, transparent 1px),
                linear-gradient(90deg, rgb(140 150 255 / 0.035) 1px, transparent 1px);
            background-size: auto, auto, 36px 36px, 36px 36px;
            overflow-x: hidden;
            -webkit-user-select: none;
            user-select: none;
            -webkit-tap-highlight-color: transparent;
        }

        button, input { touch-action: manipulation; }

        .crypt-table {
            flex: 1;
            display: flex;
            flex-direction: column;
            align-items: center;
            gap: 14px;
            padding: calc(18px + env(safe-area-inset-top, 0px)) 16px calc(24px + env(safe-area-inset-bottom, 0px));
        }

```

```
        /* Neon sign title */
        .brand {
            margin: 0;
            font-family: var(--font-display);
            font-weight: 900;
            font-size: clamp(1.05rem, 4.4vw, 1.75rem);
            letter-spacing: 0.2em;
            text-transform: uppercase;
            text-align: center;
            color: #f1e6ff;
            text-shadow: 0 0 4px rgb(var(--void-rgb) / 0.9), 0 0 16px rgb(var(--void-rgb) / 0.55);
        }
        .brand span {
            color: #e6fdff;
            text-shadow: 0 0 4px rgb(var(--you-rgb) / 0.9), 0 0 16px rgb(var(--you-rgb) / 0.55);
        }

        /* Solid panels with a thin neon edge */
        .panel {
            background: linear-gradient(180deg, rgb(21 24 51 / 0.94), rgb(12 14 30 / 0.94));
            border: 1px solid var(--line);
            border-radius: var(--radius);
            box-shadow: 0 10px 30px rgb(0 0 0 / 0.55), inset 0 1px 0 rgb(255 255 255 / 0.04);
        }

        .side {
            width: 100%;
            max-width: 64rem;
            display: flex;
            flex-direction: column;
            align-items: center;
            gap: 10px;
        }
        .side-head {
            width: 100%;
            display: flex;
            align-items: center;
            justify-content: space-between;
            gap: 8px 12px;
            padding-inline: 6px;
        }
        .side-title {
            margin: 0;
            min-width: 0;
            font-family: var(--font-display);
            font-weight: 700;
            font-size: clamp(0.92rem, 3.6vw, 1.35rem);
            letter-spacing: 0.12em;
            text-transform: uppercase;
            overflow-wrap: anywhere;
        }
        .side-foe .side-title { color: var(--foe); text-shadow: 0 0 10px rgb(var(--foe-rgb) / 0.55); }
        .side-you .side-title { color: var(--you); text-shadow: 0 0 10px rgb(var(--you-rgb) / 0.55); }

        .strength-chip {
            flex-shrink: 0;
            padding: 0.3rem 0.85rem;
            border-radius: 999px;
            border: 1px solid var(--line-strong);
            background: rgb(10 11 24 / 0.85);
            color: var(--muted);
            font-weight: 600;
            font-size: clamp(0.82rem, 3.2vw, 1rem);
            white-space: nowrap;
        }
        .strength-chip b {
            margin-left: 0.35em;
            font-family: var(--font-num);
            font-weight: 800;
            color: var(--fg);
            font-variant-numeric: tabular-nums;
        }
        .side-foe .strength-chip { border-color: rgb(var(--foe-rgb) / 0.4); }
        .side-you .strength-chip { border-color: rgb(var(--you-rgb) / 0.4); }

        .board {
            width: 100%;
            display: flex;
            justify-content: center;
            gap: 16px;
            padding: 16px;
        }
        .side-foe .board {
            border-color: rgb(var(--foe-rgb) / 0.32);
            box-shadow: 0 0 22px rgb(var(--foe-rgb) / 0.08), 0 10px 30px rgb(0 0 0 / 0.55);
        }
```

```
        .side-you .board {
            border-color: rgb(var(--you-rgb) / 0.32);
            box-shadow: 0 0 22px rgb(var(--you-rgb) / 0.08), 0 10px 30px rgb(0 0 0 / 0.55);
        }

        /* 3D Card Mechanics */
        .card-slot {
            width: var(--cw);
            height: calc(var(--cw) * 1.385);
            position: relative;
        }

        /* Random tabletop offset lives on the wrapper; animations run on .card-fx so they never erase it */
        .card-wrapper {
            position: absolute;
            inset: 0;
            perspective: 800px;
            border-radius: var(--card-radius);
            transition: transform 0.3s ease, filter 0.3s ease;
        }

        .card-fx {
            position: relative;
            width: 100%;
            height: 100%;
            border-radius: var(--card-radius);
            perspective: 1000px;
            transition: scale 0.2s ease;
        }

        .card {
            width: 100%;
            height: 100%;
            position: relative;
            transition: transform 0.6s cubic-bezier(0.4, 0.0, 0.2, 1);
            transform-style: preserve-3d;
        }

        .card.face-down {
            transform: rotateY(180deg);
        }

        .card-face {
            position: absolute;
            inset: 0;
            -webkit-backface-visibility: hidden;
            backface-visibility: hidden;
            border-radius: var(--card-radius);
            display: flex;
            flex-direction: column;
            justify-content: space-between;
            padding: calc(var(--cw) * 0.06);
            background-color: var(--card-ink);
            box-shadow: 0 6px 14px rgb(0 0 0 / 0.6);
        }

        /* --lv is the card's level colour (set on each card) */
        .card-front {
            --lv: var(--fg);
            isolation: isolate;
            border: 2px solid var(--lv);
            background: linear-gradient(180deg, #121430 0%, var(--card-ink) 100%);
            box-shadow: 0 0 16px -5px var(--lv), 0 6px 14px rgb(0 0 0 / 0.6);
        }

        /* Card back: the crypt's violet neon */
        .card-back {
            background:
                radial-gradient(circle at 50% 50%, rgb(var(--void-rgb) / 0.2), transparent 62%),
                repeating-linear-gradient(45deg, rgb(var(--void-rgb) / 0.08) 0 2px, transparent 2px 9px),
                #0c0b1e;
            border: 2px solid rgb(var(--void-rgb) / 0.6);
            box-shadow: 0 0 14px -5px var(--void), inset 0 0 0 3px #0c0b1e, inset 0 0 0 4px rgb(var(--void-rgb) / 0.25);
            transform: rotateY(180deg);
            align-items: center;
            justify-content: center;
        }

        .card-back::after {
            content: '☥';
            font-family: system-ui, sans-serif;
            font-size: calc(var(--cw) * 0.36);
            color: #dcc4ff;
            text-shadow: 0 0 6px var(--void), 0 0 16px rgb(var(--void-rgb) / 0.6);
        }

```

```
        .card-name {
            position: relative;
            z-index: 2;
            font-size: max(8px, calc(var(--cw) * 0.1));
            line-height: 1.15;
            font-weight: 700;
            letter-spacing: 0.03em;
            text-align: center;
            color: var(--lv);
            text-shadow: 0 1px 2px #000, 0 0 6px var(--lv);
        }

        .card-sigil {
            flex-grow: 1;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: calc(var(--cw) * 0.28);
            color: var(--lv);
            opacity: 0.6;
            text-shadow: 0 0 6px var(--lv);
        }

        .card-foot {
            position: relative;
            z-index: 2;
            display: flex;
            justify-content: center;
        }

        /* Creature art: full-bleed neon picture, shaded at the top and bottom so the name and points stay readable */
        .card-art {
            position: absolute;
            inset: 0;
            width: 100%;
            height: 100%;
            object-fit: cover;
            border-radius: calc(var(--card-radius) - 2px);
            pointer-events: none;
            z-index: 0;
        }

        .card-front.has-art::before {
            content: '';
            position: absolute;
            inset: 0;
            border-radius: calc(var(--card-radius) - 2px);
            background: linear-gradient(180deg,
                rgb(5 6 14 / 0.88) 0%, rgb(5 6 14 / 0) 22%,
                rgb(5 6 14 / 0) 74%, rgb(5 6 14 / 0.7) 100%);
            pointer-events: none;
            z-index: 1;
        }

        .slot-empty {
            width: 100%;
            height: 100%;
            border: 1.5px dashed rgb(140 150 255 / 0.22);
            border-radius: var(--card-radius);
            background: rgb(140 150 255 / 0.025);
        }

        /* Animations */
        /* A card settling onto the table after its flight from the crypt */
        @keyframes land {
            0% { transform: scale(1.08); filter: brightness(1.4); }
            100% { transform: none; filter: none; }
        }

        .animate-land {
            animation: land 0.28s ease-out both;
        }

        /* Card in flight from the crypt pile to its slot */
        .flyer {
            position: fixed;
            z-index: 60;
            pointer-events: none;
            perspective: 800px;
        }
        .flyer .card {
            transition-duration: 0.3s;
        }

        /* The crypt: the deck everyone draws from */
        .deck-pile {
```

```
            position: relative;
            flex-shrink: 0;
            width: 42px;
            height: 58px;
            border-radius: 7px;
            border: 2px solid rgb(var(--void-rgb) / 0.6);
            background:
                repeating-linear-gradient(45deg, rgb(var(--void-rgb) / 0.1) 0 2px, transparent 2px 7px),
                #0c0b1e;
            box-shadow:
                3px -3px 0 -1px #0c0b1e, 3px -3px 0 0 rgb(var(--void-rgb) / 0.45),
                6px -6px 0 -1px #0c0b1e, 6px -6px 0 0 rgb(var(--void-rgb) / 0.3),
                0 0 14px rgb(var(--void-rgb) / 0.25);
            display: flex;
            align-items: center;
            justify-content: center;
            transition: opacity 0.3s;
        }
        .deck-pile::after {
            content: '†';
            font-family: system-ui, sans-serif;
            font-size: 1.5rem;
            color: #dcc4ff;
            text-shadow: 0 0 6px var(--void);
        }
        .deck-pile.empty {
            opacity: 0.3;
        }

        @keyframes spectralFlame {
            0% { transform: scale(1); box-shadow: 0 0 0 rgb(var(--life-rgb) / 0); filter: brightness(1); }
            20% { transform: scale(1.15) rotate(-3deg); box-shadow: 0 0 36px rgb(var(--life-rgb) / 0.8); filter: brightnes
s(1.4); }
            40% { transform: scale(1.15) rotate(3deg); box-shadow: 0 0 56px rgb(var(--life-rgb) / 1); filter: brightness
(1.7); }
            60% { transform: scale(1.15) rotate(-3deg); box-shadow: 0 0 56px rgb(var(--life-rgb) / 1); filter: brightness
(1.7); }
            100% { transform: scale(1); box-shadow: 0 0 0 rgb(var(--life-rgb) / 0); filter: brightness(1); }
        }

        .anim-spectral {
            animation: spectralFlame 1s ease-out forwards;
        }

        @keyframes darkVoid {
            0% { transform: scale(1); filter: contrast(1); }
            30% { transform: scale(0.8) rotate(10deg); filter: contrast(1.6) hue-rotate(20deg); box-shadow: inset 0 0 50px
var(--void), 0 0 40px var(--void); }
            60% { transform: scale(0.1) rotate(-20deg); opacity: 1; }
            100% { transform: scale(0); opacity: 0; }
        }

        .anim-void {
            animation: darkVoid 0.8s ease-in forwards;
        }

        @keyframes bump {
            0% { transform: scale(1); }
            40% { transform: scale(1.35); }
            100% { transform: scale(1); }
        }

        .badge-bump {
            animation: bump 0.35s ease-out;
        }

        /* Targeting Mode: valid targets glow softly (touch screens have no hover) */
        body.targeting {
            cursor: crosshair;
        }
        body.targeting .card-wrapper {
            cursor: crosshair;
        }
        body.targeting #player-slots .card-wrapper {
            filter: drop-shadow(0 0 5px rgb(var(--life-rgb) / 0.55));
        }
        body.targeting #opponent-slots .card-wrapper {
            filter: drop-shadow(0 0 5px rgb(var(--void-rgb) / 0.6));
        }
        body.targeting .card-wrapper:hover .card-fx,
        body.targeting .card-wrapper:focus-visible .card-fx {
            scale: 1.05;
        }
        body.targeting #player-slots .card-wrapper:hover,
        body.targeting #player-slots .card-wrapper:focus-visible {
            filter: drop-shadow(0 0 12px var(--ally));
```

```
        }
        body.targeting #opponent-slots .card-wrapper:hover,
        body.targeting #opponent-slots .card-wrapper:focus-visible {
            filter: drop-shadow(0 0 12px var(--enemy));
        }

        .card-wrapper:focus-visible {
            outline: 2px solid var(--fg);
            outline-offset: 4px;
        }

        /* Matching cards joined into one pile: card edges peek out behind it */
        .card-wrapper::before,
        .card-wrapper::after {
            content: '';
            position: absolute;
            inset: 0;
            border-radius: var(--card-radius);
            border: 2px solid rgb(140 150 255 / 0.4);
            background: var(--card-ink);
            opacity: 0;
            transition: opacity 0.2s;
            pointer-events: none;
        }
        .card-wrapper::before { transform: translate(4px, -4px); z-index: -1; }
        .card-wrapper::after { transform: translate(8px, -8px); z-index: -2; }
        .card-wrapper.stack-2::before,
        .card-wrapper.stack-3::before,
        .card-wrapper.stack-3::after { opacity: 1; }
        .card-wrapper:has(.anim-void)::before,
        .card-wrapper:has(.anim-void)::after { opacity: 0; }

        /* How many matching cards are in the pile (shown from x2 up) */
        .card-count {
            position: absolute;
            top: calc(var(--cw) * -0.12);
            right: calc(var(--cw) * -0.12);
            z-index: 5;
            min-width: max(22px, calc(var(--cw) * 0.25));
            height: max(22px, calc(var(--cw) * 0.25));
            padding: 0 0.3em;
            border-radius: 999px;
            background: #0a0b18;
            border: 2px solid var(--fg);
            color: #fff;
            font-family: var(--font-num);
            font-weight: 800;
            font-size: max(11px, calc(var(--cw) * 0.11));
            font-variant-numeric: tabular-nums;
            display: flex;
            align-items: center;
            justify-content: center;
            box-shadow: 0 0 8px rgb(231 234 255 / 0.45), 0 2px 6px rgb(0 0 0 / 0.6);
        }
        .card-count[hidden] {
            display: none;
        }

        /* Points plaque: what the card gives; its colour is the creature's level */
        .card-points {
            display: inline-flex;
            align-items: baseline;
            gap: 0.25em;
            background: rgb(5 6 14 / 0.86);
            border: 1px solid var(--lv);
            border-radius: 999px;
            padding: 0.12em 0.6em;
            color: var(--lv);
            font-family: var(--font-num);
            font-size: max(12px, calc(var(--cw) * 0.15));
            font-weight: 800;
            line-height: 1.15;
            font-variant-numeric: tabular-nums;
            white-space: nowrap;
            text-shadow: 0 0 6px var(--lv);
            box-shadow: 0 0 10px -3px var(--lv);
        }
        .card-points small {
            font-family: var(--font-ui);
            font-size: 0.6em;
            letter-spacing: 0.08em;
            text-transform: uppercase;
        }

        /* Wager belt */
        .belt {
```

```
            width: 100%;
            max-width: 64rem;
            position: relative;
            z-index: 20;
            display: flex;
            flex-wrap: wrap;
            align-items: center;
            justify-content: space-between;
            gap: 14px 18px;
            padding: 14px 18px;
            border-color: rgb(var(--void-rgb) / 0.3);
        }
        .belt-info {
            display: flex;
            align-items: center;
            gap: 14px;
        }
        .round-box {
            min-width: 5.5rem;
            text-align: center;
        }
        .label {
            font-size: 0.72rem;
            font-weight: 600;
            letter-spacing: 0.14em;
            text-transform: uppercase;
            color: var(--muted);
        }
        .round-value {
            font-family: var(--font-num);
            font-weight: 800;
            font-size: 1.8rem;
            line-height: 1.15;
            color: var(--fg);
            text-shadow: 0 0 10px rgb(var(--void-rgb) / 0.6);
            font-variant-numeric: tabular-nums;
        }
        .crypt-count {
            font-size: 0.76rem;
            color: var(--muted);
        }
        .crypt-count span { font-variant-numeric: tabular-nums; }
        .divider {
            width: 1px;
            align-self: stretch;
            background: var(--line-strong);
        }
        .wager {
            display: flex;
            flex-direction: column;
            gap: 6px;
        }
        .bet-input {
            width: 9.5rem;
            padding: 0.45rem 0.75rem;
            border-radius: 10px;
            border: 1px solid rgb(var(--gold-rgb) / 0.4);
            background: #070815;
            color: var(--fg);
            font-family: var(--font-num);
            font-weight: 700;
            font-size: 1.05rem;
            transition: border-color 0.2s, box-shadow 0.2s;
        }
        .bet-input::placeholder {
            color: var(--faint);
            font-family: var(--font-ui);
            font-weight: 500;
        }
        .bet-input:focus {
            outline: none;
            border-color: var(--gold);
            box-shadow: 0 0 0 3px rgb(var(--gold-rgb) / 0.16), 0 0 14px rgb(var(--gold-rgb) / 0.3);
        }
        .bet-input:disabled { opacity: 0.55; }
        .wager-stats {
            display: flex;
            flex-wrap: wrap;
            gap: 2px 14px;
            font-size: 0.9rem;
            color: var(--muted);
        }
        .wager-stats b {
            font-family: var(--font-num);
            font-weight: 800;
            font-variant-numeric: tabular-nums;
```

```
        }
        .stat-bet b { color: var(--gold); text-shadow: 0 0 8px rgb(var(--gold-rgb) / 0.45); }
        .stat-gold b { color: var(--fg); }

        .action-bar {
            display: flex;
            flex-wrap: wrap;
            justify-content: center;
            gap: 10px;
        }

        /* Neon buttons: --tone is the button's colour */
        .btn-neon {
            --tone: var(--you-rgb);
            min-height: 46px;
            padding: 0.6rem 1.2rem;
            border-radius: 12px;
            border: 1.5px solid rgb(var(--tone) / 0.75);
            background: rgb(10 11 24 / 0.9);
            color: rgb(var(--tone));
            font-family: var(--font-display);
            font-weight: 700;
            font-size: 0.82rem;
            letter-spacing: 0.1em;
            line-height: 1.2;
            text-transform: uppercase;
            text-shadow: 0 0 8px rgb(var(--tone) / 0.5);
            box-shadow: 0 0 14px rgb(var(--tone) / 0.2), inset 0 0 14px rgb(var(--tone) / 0.08);
            transition: background 0.2s, box-shadow 0.2s, opacity 0.2s, transform 0.1s;
        }
        .btn-neon:hover:not(:disabled) {
            background: rgb(var(--tone) / 0.12);
            box-shadow: 0 0 22px rgb(var(--tone) / 0.35), inset 0 0 16px rgb(var(--tone) / 0.14);
        }
        .btn-neon:active:not(:disabled) { transform: translateY(1px); }
        .btn-neon:focus-visible {
            outline: 2px solid rgb(var(--tone));
            outline-offset: 3px;
        }
        .btn-neon:disabled {
            opacity: 0.35;
            cursor: not-allowed;
            box-shadow: none;
            text-shadow: none;
        }
        .btn-main { --tone: var(--you-rgb); background: rgb(var(--you-rgb) / 0.1); }
        .btn-life { --tone: var(--life-rgb); }
        .btn-danger { --tone: var(--danger-rgb); }

        /* How to play: tucked away until wanted */
        .rules {
            width: 100%;
            max-width: 42rem;
            border: 1px solid var(--line);
            border-radius: 12px;
            background: rgb(12 14 30 / 0.75);
        }
        .rules summary {
            display: flex;
            align-items: center;
            justify-content: space-between;
            min-height: 44px;
            padding: 0.6rem 1rem;
            cursor: pointer;
            list-style: none;
            font-family: var(--font-display);
            font-weight: 700;
            font-size: 0.75rem;
            letter-spacing: 0.14em;
            text-transform: uppercase;
            color: var(--muted);
        }
        .rules summary::-webkit-details-marker { display: none; }
        .rules summary::after {
            content: '+';
            font-size: 1.15rem;
            color: var(--you);
            text-shadow: 0 0 8px rgb(var(--you-rgb) / 0.6);
        }
        .rules[open] summary::after { content: '−'; }
        .rules summary:focus-visible {
            outline: 2px solid var(--you);
            outline-offset: -2px;
            border-radius: 12px;
        }
        .rules p {
```

```
            margin: 0;
            padding: 0 1rem 1rem;
            color: var(--muted);
            font-size: 0.92rem;
            line-height: 1.6;
            text-wrap: pretty;
        }
        .hl-life { color: var(--ally); }
        .hl-void { color: var(--enemy); }
        .hl-gold { color: var(--gold); }

        /* Colour key under the rules */
        .level-legend {
            display: flex;
            flex-wrap: wrap;
            justify-content: center;
            gap: 0.4rem 1rem;
            margin: 0;
            padding: 0 0.5rem;
            list-style: none;
            font-size: 0.8rem;
            color: var(--muted);
        }
        .level-legend li {
            display: inline-flex;
            align-items: center;
            gap: 0.4rem;
            white-space: nowrap;
        }
        .level-legend b {
            font-family: var(--font-num);
            font-weight: 800;
            font-variant-numeric: tabular-nums;
            color: var(--lv);
            text-shadow: 0 0 6px var(--lv);
        }
        .legend-swatch {
            width: 0.7rem;
            height: 0.7rem;
            border-radius: 3px;
            background: var(--lv);
            box-shadow: 0 0 8px var(--lv);
        }

        #toast {
            position: fixed;
            left: 50%;
            bottom: calc(var(--bar-h) + 16px + env(safe-area-inset-bottom, 0px));
            z-index: 50;
            transform: translateX(-50%);
            width: max-content;
            max-width: min(32rem, calc(100vw - 32px));
            padding: 0.7rem 1.15rem;
            border-radius: 12px;
            border: 1px solid rgb(var(--you-rgb) / 0.45);
            background: rgb(15 17 36 / 0.97);
            box-shadow: 0 0 18px rgb(var(--you-rgb) / 0.16), 0 10px 30px rgb(0 0 0 / 0.6);
            color: var(--fg);
            font-weight: 600;
            font-size: 1rem;
            text-align: center;
            text-wrap: balance;
            opacity: 0;
            pointer-events: none;
            transition: opacity 0.3s;
        }

        /* Crypt Fortune: the luck roll after a win */
        .fortune {
            position: fixed;
            inset: 0;
            z-index: 70;
            display: flex;
            align-items: center;
            justify-content: center;
            padding: 16px;
            background: rgb(4 5 12 / 0.78);
        }
        .fortune[hidden] {
            display: none;
        }
        .fortune-panel {
            width: min(380px, 100%);
            padding: 1.5rem 1.25rem;
            text-align: center;
            border-color: rgb(var(--void-rgb) / 0.4);
```

```
765             box-shadow: 0 0 30px rgb(var(--void-rgb) / 0.18), 0 10px 30px rgb(0 0 0 / 0.6);
766         }
767         .fortune-title {
768             font-family: var(--font-display);
769             font-size: 0.75rem;
770             font-weight: 700;
771             letter-spacing: 0.22em;
772             text-transform: uppercase;
773             color: var(--muted);
774         }
775         .fortune-reel {
776             width: 7.5rem;
777             height: 4.5rem;
778             margin: 1rem auto;
779             display: flex;
780             align-items: center;
781             justify-content: center;
782             border-radius: 12px;
783             border: 2px solid var(--line-strong);
784             background: #05060e;
785             color: var(--fg);
786             font-family: var(--font-num);
787             font-size: 2.5rem;
788             font-weight: 800;
789             font-variant-numeric: tabular-nums;
790             transition: color 0.3s, border-color 0.3s, box-shadow 0.3s;
791         }
792         .fortune-headline {
793             min-height: 1.4em;
794             font-family: var(--font-display);
795             font-size: 1.45rem;
796             font-weight: 900;
797             letter-spacing: 0.08em;
798             text-wrap: balance;
799         }
800         .fortune-result {
801             min-height: 2.8em;
802             margin-top: 0.4rem;
803             color: var(--fg);
804             text-wrap: balance;
805         }
806         .fortune.tier-lucky .fortune-reel { color: var(--ally); border-color: var(--ally); box-shadow: 0 0 24px rgb(var(--
    life-rgb) / 0.55); }
807         .fortune.tier-lucky .fortune-headline { color: var(--ally); text-shadow: 0 0 12px rgb(var(--life-rgb) / 0.75); }
808         .fortune.tier-big .fortune-reel { color: var(--gold); border-color: var(--gold); box-shadow: 0 0 34px rgb(var(--go
    ld-rgb) / 0.7); }
809         .fortune.tier-big .fortune-headline { color: var(--gold); text-shadow: 0 0 16px rgb(var(--gold-rgb) / 0.85); }
810         .fortune.tier-mega .fortune-reel { color: var(--foe); border-color: var(--foe); box-shadow: 0 0 44px rgb(var(--foe
    -rgb) / 0.8), 0 0 90px rgb(var(--void-rgb) / 0.6); }
811         .fortune.tier-mega .fortune-headline { color: var(--foe); text-shadow: 0 0 18px rgb(var(--foe-rgb) / 0.85), 0 0 36
    px rgb(var(--void-rgb) / 0.85); }
812         .fortune.tier-big .fortune-panel,
813         .fortune.tier-mega .fortune-panel {
814             animation: fortunePulse 0.9s ease-in-out 3;
815         }
816         @keyframes fortunePulse {
817             0%, 100% { transform: scale(1); }
818             50% { transform: scale(1.06); }
819         }
820
821         /* Duel: join screen, opponent status and planned spells */
822         .side-names {
823             display: flex;
824             flex-wrap: wrap;
825             align-items: center;
826             gap: 4px 10px;
827             min-width: 0;
828         }
829         .join-form { text-align: left; }
830         .join-form .fortune-title { text-align: center; }
831         .join-heading {
832             margin: 0.5rem 0 1.1rem;
833             text-align: center;
834             font-family: var(--font-display);
835             font-size: 1.45rem;
836             font-weight: 900;
837             letter-spacing: 0.08em;
838             text-transform: uppercase;
839             color: #e6fdff;
840             text-shadow: 0 0 6px rgb(var(--you-rgb) / 0.8), 0 0 18px rgb(var(--you-rgb) / 0.4);
841         }
842         .join-form label {
843             display: block;
844             margin-bottom: 0.35rem;
845             font-size: 0.85rem;
846             font-weight: 600;
```

```
            color: var(--muted);
        }
        .join-input {
            width: 100%;
            min-height: 46px;
            padding: 0.5rem 0.8rem;
            border: 1px solid rgb(var(--you-rgb) / 0.4);
            border-radius: 10px;
            background: #070815;
            color: var(--fg);
            font: inherit;
            font-size: 16px;
            font-weight: 600;
        }
        .join-input:focus {
            outline: none;
            border-color: var(--you);
            box-shadow: 0 0 0 3px rgb(var(--you-rgb) / 0.16), 0 0 14px rgb(var(--you-rgb) / 0.3);
        }
        .join-button {
            width: 100%;
            margin-top: 1rem;
        }
        .join-error {
            min-height: 1.4em;
            margin: 0.75rem 0 0;
            color: var(--blood);
            font-size: 0.9rem;
        }
        .opp-status {
            padding: 0.18rem 0.65rem;
            border: 1px solid var(--line-strong);
            border-radius: 999px;
            background: rgb(10 11 24 / 0.85);
            color: var(--muted);
            font-size: 0.78rem;
            font-weight: 600;
            letter-spacing: 0.04em;
            white-space: nowrap;
        }
        .opp-status.is-ready { color: var(--ally); border-color: rgb(var(--life-rgb) / 0.7); box-shadow: 0 0 10px rgb(var
(--life-rgb) / 0.25); }
        .opp-status.is-away { color: var(--blood); border-color: rgb(var(--danger-rgb) / 0.7); }
        .card-front-unknown {
            border: 2px solid rgb(var(--void-rgb) / 0.6);
            background: #0c0b1e;
        }
        .plan-tag {
            position: absolute;
            left: 50%;
            top: calc(100% + 4px);
            z-index: 6;
            transform: translateX(-50%);
            padding: 0.12em 0.6em;
            border: 1px solid;
            border-radius: 999px;
            background: #070815;
            font-size: max(9px, calc(var(--cw) * 0.085));
            font-weight: 700;
            letter-spacing: 0.04em;
            white-space: nowrap;
            pointer-events: none;
        }
        .plan-double .card-wrapper { outline: 2px solid var(--ally); outline-offset: 4px; box-shadow: 0 0 18px rgb(var(--l
ife-rgb) / 0.45); }
        .plan-double .plan-tag { color: var(--ally); border-color: var(--ally); box-shadow: 0 0 10px rgb(var(--life-rgb) /
0.35); }
        .plan-implode .card-wrapper { outline: 2px dashed var(--enemy); outline-offset: 4px; box-shadow: 0 0 18px rgb(var
(--void-rgb) / 0.45); }
        .plan-implode .plan-tag { color: var(--enemy); border-color: var(--enemy); box-shadow: 0 0 10px rgb(var(--void-rg
b) / 0.35); }

        /* Phones: tighter edges, bigger cards, buttons in a bar at the bottom */
        @media (max-width: 640px) {
            :root {
                --cw: calc((100vw - 62px) / 5);
                --bar-h: 68px;
                --radius: 12px;
                --card-radius: 8px;
            }
            .crypt-table {
                gap: 10px;
                padding: calc(12px + env(safe-area-inset-top, 0px)) 10px calc(var(--bar-h) + 18px + env(safe-area-inset-bo
ttom, 0px));
            }
            .side { gap: 6px; }
```

```
            .side-head { padding-inline: 2px; }
            .board { gap: 6px; padding: 9px 6px; }
            .belt { padding: 10px 12px; gap: 10px; }
            .belt-info { width: 100%; justify-content: space-between; gap: 10px; }
            .deck-pile { width: 34px; height: 47px; }
            .deck-pile::after { font-size: 1.2rem; }
            .round-box { min-width: 0; }
            .round-value { font-size: 1.35rem; }
            .bet-input { width: 100%; max-width: 9rem; font-size: 16px; } /* 16px stops iOS zooming in */
            .wager-stats { font-size: 0.82rem; gap: 0 12px; }
            .card-points small { display: none; } /* Phone cards are narrow: the number alone */
            .action-bar {
                position: fixed;
                left: 0;
                right: 0;
                bottom: 0;
                z-index: 40;
                flex-wrap: nowrap;
                gap: 8px;
                padding: 10px 10px calc(10px + env(safe-area-inset-bottom, 0px));
                background: rgb(7 8 18 / 0.95);
                border-top: 1px solid rgb(var(--void-rgb) / 0.32);
                box-shadow: 0 -10px 30px rgb(0 0 0 / 0.6);
                -webkit-backdrop-filter: blur(10px);
                backdrop-filter: blur(10px);
            }
            .action-bar .btn-neon {
                flex: 1 1 0;
                min-width: 0;
                min-height: 48px;
                padding: 0.45rem 0.35rem;
                font-size: 0.66rem;
                letter-spacing: 0.06em;
            }
            #toast { font-size: 0.95rem; }
            .card-count { top: -11px; right: -8px; } /* Clear of the card name on narrow cards */
            .rules p { font-size: 0.9rem; }
            .level-legend { font-size: 0.74rem; gap: 0.35rem 0.8rem; }
        }

        /* Phones held sideways: cards sized by the screen's height, so both armies and the buttons fit */
        @media (orientation: landscape) and (max-height: 520px) {
            :root {
                --cw: min(110px, calc((100vh - 190px) / 2.77), calc((100vw - 120px) / 5));
                --bar-h: 0px;
            }
            @supports (height: 100dvh) {
                :root { --cw: min(110px, calc((100dvh - 190px) / 2.77), calc((100vw - 120px) / 5)); }
            }
            .brand { display: none; }
            .crypt-table {
                gap: 8px;
                padding: 8px max(16px, env(safe-area-inset-right, 0px)) calc(12px + env(safe-area-inset-bottom, 0px)) max
(16px, env(safe-area-inset-left, 0px));
            }
            .side { gap: 4px; }
            .side-title { font-size: 0.9rem; }
            .strength-chip { padding: 0.15rem 0.7rem; font-size: 0.82rem; }
            .board { gap: 10px; padding: 6px; }
            .belt { padding: 6px 12px; gap: 8px 14px; }
            .belt-info { width: auto; gap: 12px; }
            .deck-pile { width: 30px; height: 41px; }
            .deck-pile::after { font-size: 1.05rem; }
            .round-value { font-size: 1.2rem; }
            .label, .crypt-count { font-size: 0.68rem; }
            .bet-input { width: 7.5rem; padding: 0.3rem 0.6rem; }
            .wager { gap: 3px; }
            .wager-stats { font-size: 0.8rem; }
            .action-bar {
                position: static;
                flex-wrap: nowrap;
                gap: 8px;
                padding: 0;
                background: none;
                border: 0;
                box-shadow: none;
                -webkit-backdrop-filter: none;
                backdrop-filter: none;
            }
            .action-bar .btn-neon {
                flex: 0 1 auto;
                min-height: 40px;
                padding: 0.4rem 0.8rem;
                font-size: 0.68rem;
                letter-spacing: 0.06em;
            }
```

```
            .card-points small { display: none; }
            #toast { top: 8px; bottom: auto; } /* Over the opponent's title, clear of your own cards */
        }

        @media (prefers-reduced-motion: reduce) {
            .card, .card-wrapper, .card-fx { transition-duration: 0.01ms !important; }
            .animate-land, .anim-spectral, .anim-void, .badge-bump, .fortune-panel {
                animation-duration: 0.01ms !important;
                animation-delay: 0ms !important;
            }
        }
    </style>
    <!-- build:tailwind (precompiled Tailwind CSS, generated by build.py) -->
```

[lines 1025–1025 omitted: generated Tailwind CSS (third-party, MIT licence) / გენერირებული Tailwind CSS]

```
    <!-- /build:tailwind -->
</head>
<body>
<main class="crypt-table">

    <h1 class="brand">Necropolis <span>Duel</span></h1>

    <!-- Opponent Board (Top) -->
    <section class="side side-foe" aria-label="Opponent board">
        <div class="side-head">
            <div class="side-names">
                <h2 id="opp-name" class="side-title">Waiting for an opponent</h2>
                <span id="opp-status" class="opp-status" hidden></span>
            </div>
            <div class="strength-chip">Strength <b id="opp-strength">???</b></div>
        </div>
        <div id="opponent-slots" class="board panel">
            <!-- Opponent Cards Injected Here -->
        </div>
    </section>

    <!-- Middle Belt (Controls & Decks) -->
    <section class="belt panel" aria-label="Wagers and actions">
        <div class="belt-info">
            <div id="deck-pile" class="deck-pile" role="img" aria-label="The crypt: the deck both armies draw from"></div>
            <div class="round-box">
                <div class="label">Round</div>
                <div id="round-display" class="round-value">1 / 3</div>
                <div class="crypt-count"><span id="deck-display">59</span> in the crypt</div>
            </div>
            <div class="divider"></div>
            <div class="wager">
                <label for="bet-input" class="sr-only">Bet amount in gold</label>
                <input type="number" id="bet-input" class="bet-input" min="0" step="1" inputmode="numeric" placeholder="Be
t amount">
                <div class="wager-stats">
                    <span class="stat-bet">Your bet <b id="pot-display">0</b></span>
                    <span class="stat-gold">Gold <b id="gold-display">1000</b></span>
                </div>
            </div>
        </div>

        <div class="action-bar">
            <button id="btn-draw" class="btn-neon btn-main" disabled>Waiting for Opponent</button>
            <button id="btn-spell" class="btn-neon btn-life" disabled>Cast Spell (1)</button>
            <button id="btn-resolve" class="btn-neon btn-danger" disabled>Ready</button>
        </div>
    </section>

    <!-- Player Board (Bottom) -->
    <section class="side side-you" aria-label="Your board">
        <div id="player-slots" class="board panel">
            <!-- Player Cards Injected Here -->
        </div>
        <div class="side-head">
            <h2 id="you-name" class="side-title">Your Army</h2>
            <div class="strength-chip">Strength <b id="player-strength">0</b></div>
        </div>
    </section>

    <details class="rules">
        <summary>How to play &amp;amp; payouts</summary>
        <p>
            Duel rules: both players bet once, before round 1. Each round both armies draw five cards from the crypt, deal
t in pairs of similar value.
            Each card shows the points it gives, and its colour shows its level; matching cards join into one pile (x2, x
3).
            Your opponent's cards stay face down until the end of each round. Each round you secretly plan one spell:
            <span class="hl-life">Animate Dead</span> doubles one of your cards and upgrades it,
            <span class="hl-void">Implosion</span> destroys one of theirs. Press Ready; when both players are ready, both
spells take effect together
            (a card that is doubled and destroyed in the same round is destroyed) and both armies are revealed.
```

```
        </p>
        <p>
            After three rounds the stronger army wins: the winner gets their bet back plus winnings of bet &times; 0.20 &times; (their points &divide; 100),
            and the <span class="hl-gold">Crypt Fortune</span> may multiply those winnings (Lucky &times;2, BIG &times;10, MEGA &times;50). The loser's bet goes to the Necropolis; a draw returns both bets.
        </p>
    </details>
    <ul id="level-legend" class="level-legend" aria-label="Card colour for each level"></ul>

</main>

    <!-- Crypt Fortune: luck roll after a win -->
    <div id="fortune" class="fortune" hidden>
        <div class="fortune-panel panel" role="status" aria-live="polite">
            <div class="fortune-title">Crypt Fortune</div>
            <div id="fortune-reel" class="fortune-reel">×1</div>
            <div id="fortune-headline" class="fortune-headline"></div>
            <div id="fortune-result" class="fortune-result"></div>
        </div>
    </div>

    <!-- Take a seat at the table -->
    <div id="join" class="fortune" hidden>
        <form id="join-form" class="fortune-panel panel join-form">
            <div class="fortune-title">Necropolis Casino Duel</div>
            <h1 class="join-heading">Take a seat</h1>
            <label for="join-name">Your name</label>
            <input id="join-name" class="join-input" maxlength="20" autocomplete="nickname" required>
            <button type="submit" class="btn-neon btn-main join-button">Join the Table</button>
            <p id="join-error" class="join-error" role="alert"></p>
        </form>
    </div>

    <!-- Toast Notification -->
    <div id="toast" role="status" aria-live="polite">
        Notification
    </div>

    <!-- build:card-art (pictures from img/ as data URIs, generated by build.py) -->
[ lines 1132-1149 omitted: 14 embedded card pictures (WebP, base64) - the same files as img/*.webp / ჩაშენებული ბარათების სურათები ]
    <!-- /build:card-art -->

    <script>
        // --- GAME DATA (cards look and score exactly as in the single-player game) ---
        const UNIT_DATA = [
            { name: 'Skeleton', level: 1, spawn: 20, power: 1, count: 20, color: '#cfd8ff', art: 'skeleton' },       // Ice white: 20 pts
            { name: 'Zombie', level: 2, spawn: 12, power: 2, count: 12, color: '#4dff88', art: 'zombie' },           // Green: 24 pts
            { name: 'Wight', level: 3, spawn: 9, power: 3, count: 9, color: '#38d6ff', art: 'wight' },               // Sky blue: 27 pts
            { name: 'Vampire', level: 4, spawn: 8, power: 4, count: 8, color: '#c77dff', art: 'vampire' },           // Violet: 32 pts
            { name: 'Lich', level: 5, spawn: 6, power: 7, count: 6, color: '#ff9a3c', art: 'lich' },                 // Orange: 42 pts
            { name: 'Dark Knight', level: 6, spawn: 3, power: 17, count: 3, color: '#ff4d6d', art: 'dark-knight' },  // Red: 51 pts
            { name: 'Bone Dragon', level: 7, spawn: 1, power: 60, count: 1, color: '#ffe047', art: 'bone-dragon' }   // Gold: 60 pts
        ];
        UNIT_DATA.forEach(unit => { unit.pts = unit.spawn * unit.power; }); // Points one card gives
        const UNIT_BY_NAME = Object.fromEntries(UNIT_DATA.map(unit => [unit.name, unit]));

        function artSrc(unit, empowered) {
            // Embedded copy first; falls back to the img/ file if the page hasn't been built
            const name = `${unit.art}${empowered ? '-empowered' : ''}`;
            return CARD_ART[name] || `../img/${name}.webp`;
        }

        // Decode the pictures up front so dealt cards and empowered swaps never show up blank
        UNIT_DATA.forEach(unit => {
            new Image().src = artSrc(unit, false);
            new Image().src = artSrc(unit, true);
        });

        const SLOT_COUNT = 5;
        const TOTAL_ROUNDS = 3;
        const STARTING_GOLD = 1000;
        const DEFAULT_BET = 100;
        // Crypt Fortune tiers (the server rolls them; these are for the banner)
        const FORTUNE_TIERS = [
            { name: 'MEGA WIN', key: 'mega', multiplier: 50 },
            { name: 'BIG WIN', key: 'big', multiplier: 10 },
            { name: 'Lucky Win', key: 'lucky', multiplier: 2 },
```

```
        ];
        const FORTUNE_FACES = [1, 2, 10, 50];

        const REDUCE_MOTION = window.matchMedia?.('(prefers-reduced-motion: reduce)').matches ?? false;
        const FLIGHT_MS = REDUCE_MOTION ? 0 : 340;       // One card flying from the crypt to the table
        const DEAL_STAGGER_MS = REDUCE_MOTION ? 0 : 130; // The second card of a pair leaves just after the first

        // --- Local view of the table: the server is the source of truth, this only mirrors it ---
        let playerArmy = [];
        let opponentArmy = [];
        let currentRound = 0;
        let totalPot = 0;
        let deckCount = 59;
        let phase = 'connecting'; // lobby, betting, spells, reveal, finished
        let fogOfWar = true;
        let gameId = 0;
        let me = null;            // { name, bet, ready, spell }
        let opp = null;           // { name, connected, betPlaced, ready, spellChosen }
        let isTargeting = false;
        let busy = false;         // true while animations play
        let skipArmedUntil = 0;   // Ready without a spell needs a second click
        let token = null;
        let myName = '';
        let gold = STARTING_GOLD;
        let pollLoop = 0;         // Bumped when a new listening loop replaces the old one

        // --- DOM ELEMENTS ---
        const els = {
            oppSlots: document.getElementById('opponent-slots'),
            playerSlots: document.getElementById('player-slots'),
            oppStrength: document.getElementById('opp-strength'),
            playerStrength: document.getElementById('player-strength'),
            oppName: document.getElementById('opp-name'),
            oppStatus: document.getElementById('opp-status'),
            youName: document.getElementById('you-name'),
            btnDraw: document.getElementById('btn-draw'),
            btnSpell: document.getElementById('btn-spell'),
            btnResolve: document.getElementById('btn-resolve'),
            roundDisp: document.getElementById('round-display'),
            potDisp: document.getElementById('pot-display'),
            goldDisp: document.getElementById('gold-display'),
            deckDisp: document.getElementById('deck-display'),
            deckPile: document.getElementById('deck-pile'),
            betInput: document.getElementById('bet-input'),
            toast: document.getElementById('toast'),
            fortune: document.getElementById('fortune'),
            fortuneReel: document.getElementById('fortune-reel'),
            fortuneHeadline: document.getElementById('fortune-headline'),
            fortuneResult: document.getElementById('fortune-result'),
            join: document.getElementById('join'),
            joinForm: document.getElementById('join-form'),
            joinName: document.getElementById('join-name'),
            joinError: document.getElementById('join-error'),
            legend: document.getElementById('level-legend'),
            body: document.body
        };

        // Colour key: which colour is which level, and the points one card gives
        els.legend.innerHTML = UNIT_DATA.map(unit =>
            `<li style="--lv: ${unit.color};"><span class="legend-swatch"></span>Lv ${unit.level} ${unit.name} <b>${unit.p
ts}</b></li>`
        ).join('');

        // --- STORAGE: gold is kept per player name in this browser; the seat token per tab ---
        const store = {
            get(area, key) { try { return window[area].getItem(key); } catch (e) { return null; } },
            set(area, key, value) { try { window[area].setItem(key, value); } catch (e) { /* storage blocked */ } },
        };
        const goldKey = () => `necropolis-duel-gold:${myName}`;
        function loadGold() {
            const saved = parseInt(store.get('localStorage', goldKey()), 10);
            return Number.isFinite(saved) && saved > 0 ? saved : STARTING_GOLD;
        }
        function saveGold() {
            store.set('localStorage', goldKey(), String(gold));
        }
        // Each game's payout is credited once, even if the page reconnects
        function markPaid(id) {
            if (store.get('sessionStorage', 'necropolis-duel-paid') === String(id)) return false;
            store.set('sessionStorage', 'necropolis-duel-paid', String(id));
            return true;
        }

        let toastTimer = null;
        function showToast(msg, duration = 3000) {
            clearTimeout(toastTimer);
```

```
            els.toast.textContent = msg;
            els.toast.style.opacity = '1';
            toastTimer = setTimeout(() => { els.toast.style.opacity = '0'; }, duration);
        }

        // Points a card on the table gives: matching cards add up, Animate Dead doubles them
        function stackPoints(stack) {
            return stack && stack.pts ? stack.pts * stack.qty : 0;
        }
        function getArmyStrength(army) {
            return army.reduce((total, stack) => total + stackPoints(stack), 0);
        }
        function wait(ms) {
            return new Promise(resolve => setTimeout(resolve, ms));
        }

        // A pile from the server plus the look of its creature (unknown enemy piles stay blank)
        function decorate(pile) {
            if (!pile) return null;
            return pile.name ? { ...UNIT_BY_NAME[pile.name], ...pile, known: true } : { ...pile, known: false };
        }

        // --- TALKING TO THE SERVER ---
        // Skips the warning page ngrok's free tunnels put in front of a site
        const TUNNEL_HEADERS = { 'ngrok-skip-browser-warning': '1' };

        async function post(path, body) {
            const res = await fetch(path, { method: 'POST', headers: { ...TUNNEL_HEADERS, 'Content-Type': 'application/jso
n' }, body: JSON.stringify(body) });
            const data = await res.json().catch(() => ({}));
            if (!res.ok) throw new Error(data.error || 'The server did not answer.');
            return data;
        }
        async function sendAction(action) {
            try {
                await post('action', { token, ...action });
                return true;
            } catch (e) {
                showToast(e.message);
                return false;
            }
        }

        async function joinTable(name, savedToken) {
            const data = await post('join', { name, token: savedToken });
            token = data.token;
            myName = name;
            store.set('sessionStorage', 'necropolis-duel-token', token);
            store.set('localStorage', 'necropolis-duel-name', name);
            gold = loadGold();
            els.join.hidden = true;
            listenToServer();
        }

        // Keeps asking the server for news; each question waits up to 25 seconds for something to happen
        async function listenToServer() {
            const loop = ++pollLoop;
            let after = -1;
            let failures = 0;
            while (loop === pollLoop) {
                try {
                    const res = await fetch(`poll?token=${encodeURIComponent(token)}&after=${after}`, { cache: 'no-store',
headers: TUNNEL_HEADERS });
                    if (loop !== pollLoop) return;
                    if (res.status === 401) {
                        // The server forgot this seat (it was restarted): join again
                        els.joinError.textContent = 'Lost the table. Join again to keep playing.';
                        els.join.hidden = false;
                        return;
                    }
                    if (!res.ok) throw new Error(`The server answered ${res.status}`);
                    const data = await res.json();
                    if (loop !== pollLoop) return;
                    failures = 0;
                    after = data.seq;
                    for (const message of data.messages) enqueue(message);
                } catch (e) {
                    if (loop !== pollLoop) return;
                    if (failures++ % 5 === 0) showToast('Connection lost. Reconnecting…', 2500);
                    await new Promise(resolve => setTimeout(resolve, 2000));
                }
            }
        }

        els.joinForm.addEventListener('submit', async (e) => {
            e.preventDefault();
```

```
            const name = els.joinName.value.trim();
            if (!name) { els.joinError.textContent = 'Enter a name so your opponent knows who you are.'; return; }
            els.joinError.textContent = '';
            try {
                await joinTable(name, store.get('sessionStorage', 'necropolis-duel-token'));
            } catch (err) {
                els.joinError.textContent = err.message;
            }
        });

        // Server messages are played one after another so animations never overlap
        const queue = [];
        let draining = false;
        function enqueue(message) {
            queue.push(message);
            if (!draining) drain();
        }
        async function drain() {
            draining = true;
            while (queue.length) {
                const message = queue.shift();
                try { await handleMessage(message); } catch (err) { console.error(err); }
            }
            draining = false;
        }
        async function handleMessage(m) {
            if (m.type === 'state') applyState(m);
            else if (m.type === 'deal') await animateDeal(m);
            else if (m.type === 'resolve') await animateResolve(m);
            else if (m.type === 'gameover') await showGameOver(m);
            else if (m.type === 'info') showToast(m.text, 4000);
        }

        function applyState(s) {
            const isNewGame = s.gameId !== gameId;
            gameId = s.gameId;
            phase = s.phase;
            currentRound = s.round;
            deckCount = s.deckCount;
            fogOfWar = s.fog;
            me = s.you;
            opp = s.opp;
            totalPot = s.you.bet;
            playerArmy = s.you.army.map(decorate);
            opponentArmy = s.opp ? s.opp.army.map(decorate) : [];

            if (phase === 'betting' && isNewGame) {
                if (gold <= 0) {
                    gold = STARTING_GOLD;
                    saveGold();
                    showToast(`Your purse is empty. The Necropolis lends you ${STARTING_GOLD} gold.`);
                }
                els.betInput.value = Math.min(DEFAULT_BET, gold);
            }
            // A result missed while disconnected is still paid out
            if (phase === 'finished' && s.result && markPaid(gameId)) {
                gold += s.result.collected;
                saveGold();
                showToast(`Result: you ${s.result.you} vs ${s.result.opp}. You collect ${s.result.collected} gold.`, 500
0);
            }
            if (phase !== 'spells' || me.ready) setTargeting(false);
            updateUI();
        }

        // --- DEALING ---
        async function animateDeal(m) {
            busy = true;
            updateUI();
            for (const pair of m.pairs) {
                const [lead, follow] = pair.steps;
                for (const card of lead.discards) {
                    deckCount -= 1;
                    updateDeck();
                    await flyCard(decorate(card), lead.side === 'opp', null);
                    if (lead.side === 'you') showToast(`No room for a ${card.name}: it crumbles to dust.`, 1800);
                }
                await Promise.all([
                    lead.landing ? landStep(lead) : null,
                    follow && follow.landing ? wait(DEAL_STAGGER_MS).then(() => landStep(follow)) : null
                ]);
            }
            busy = false;
            updateUI();
        }

```

```
        // Flies a dealt card to its slot and adds it to the local army
        async function landStep(step) {
            const isOpponent = step.side === 'opp';
            const army = isOpponent ? opponentArmy : playerArmy;
            const { index, joins, card, pile } = step.landing;
            const spot = joins ? army[index] : pile;
            deckCount -= 1;
            updateDeck();
            await flyCard(card ? decorate(card) : {}, isOpponent, { index, rot: spot.rot, tx: spot.tx, ty: spot.ty });
            if (joins) army[index].qty += 1;
            else army[index] = decorate(pile);
            updateUI();
        }

        // Animates one card from the crypt pile to a slot (or off the table when it is discarded)
        function flyCard(card, isOpponent, target) {
            const slots = isOpponent ? els.oppSlots : els.playerSlots;
            const slotRect = slots.children[target ? target.index : Math.floor(SLOT_COUNT / 2)].getBoundingClientRect();
            const pileRect = els.deckPile.getBoundingClientRect();
            const left = pileRect.left + pileRect.width / 2 - slotRect.width / 2;
            const top = pileRect.top + pileRect.height / 2 - slotRect.height / 2;

            const flyer = document.createElement('div');
            flyer.className = 'flyer';
            flyer.style.cssText = `left: ${left}px; top: ${top}px; width: ${slotRect.width}px; height: ${slotRect.height}p
x;`;
            // Your cards turn face up on the way; your opponent's stay hidden
            flyer.innerHTML = `<div class="card face-down">${isOpponent ? '' : cardFrontHTML({ ...card, qty: 1 })}<div cla
ss="card-face card-back"></div></div>`;
            document.body.appendChild(flyer);
            if (!isOpponent) {
                requestAnimationFrame(() => flyer.querySelector('.card').classList.remove('face-down'));
            }

            const dx = slotRect.left - left;
            const dy = slotRect.top - top;
            const from = `translate(0, 0) scale(${pileRect.width / slotRect.width}) rotate(-12deg)`;
            const frames = target
                ? [{ transform: from }, { transform: `translate(${dx + target.tx}px, ${dy + target.ty}px) scale(1) rotate
(${target.rot}deg)` }]
                : [{ transform: from, opacity: 1 },
                   { transform: `translate(${dx}px, ${dy}px) scale(0.9) rotate(6deg)`, opacity: 1, offset: 0.65 },
                   { transform: `translate(${dx}px, ${dy - 30}px) scale(0.4) rotate(24deg)`, opacity: 0 }];
            const flight = flyer.animate(frames, {
                duration: target ? FLIGHT_MS : FLIGHT_MS * 2,
                easing: 'cubic-bezier(0.25, 0.8, 0.3, 1)'
            });
            return flight.finished.catch(() => {}).then(() => flyer.remove());
        }

        // --- SPELLS: each player secretly plans one, both take effect when both are ready ---
        function canPlanSpell() {
            return phase === 'spells' && me && !me.ready && !busy;
        }
        function spellLabel() {
            if (isTargeting) return 'Cancel Target';
            return me && me.spell ? 'Change Spell' : 'Cast Spell (1)';
        }
        function setTargeting(on) {
            isTargeting = on;
            els.body.classList.toggle('targeting', on);
            els.btnSpell.textContent = spellLabel();
        }

        els.btnSpell.addEventListener('click', () => {
            if (!canPlanSpell()) return;
            setTargeting(!isTargeting);
            if (isTargeting) showToast('Click one of your cards to double it, or an enemy card to destroy it. Click your p
lanned card again to clear it.', 4500);
        });

        document.addEventListener('keydown', (e) => {
            if (e.key === 'Escape' && isTargeting) setTargeting(false);
        });

        async function handleCardClick(isOpponent, index) {
            if (!isTargeting || !canPlanSpell()) return;
            const target = (isOpponent ? opponentArmy : playerArmy)[index];
            if (!target || target.qty === 0) return;
            const type = isOpponent ? 'implode' : 'double';
            const clearing = me.spell && me.spell.type === type && me.spell.index === index;
            setTargeting(false);
            const ok = await sendAction({ type: 'spell', spell: clearing ? null : { type, index } });
            if (!ok) return;
            if (clearing) showToast('Spell cleared.');
            else showToast(type === 'double'
```

```
                ? `Animate Dead planned on your ${target.name}. Press Ready when you're done.`
                : 'Implosion planned on an enemy card. Press Ready when you\'re done.', 3500);
        }

        els.btnResolve.addEventListener('click', async () => {
            if (!canPlanSpell()) return;
            if (!me.spell && Date.now() > skipArmedUntil) {
                skipArmedUntil = Date.now() + 4000;
                showToast("You haven't planned a spell. Press Ready again to skip it this round.", 4000);
                return;
            }
            setTargeting(false);
            await sendAction({ type: 'ready' });
        });

        // Plays a spell animation and resolves when it finishes
        function playSpell(elementId, animClass) {
            return new Promise(resolve => {
                const el = document.getElementById(elementId);
                if (!el) {
                    resolve();
                    return;
                }
                let done = false;
                const finish = () => {
                    if (done) return;
                    done = true;
                    el.removeEventListener('animationend', onEnd);
                    // A voided card stays collapsed until the board re-renders without it
                    if (animClass !== 'anim-void') el.classList.remove(animClass);
                    resolve();
                };
                const onEnd = (e) => { if (e.target === el) finish(); };

                el.classList.remove('animate-land');
                void el.offsetWidth; // Restart animation cleanly
                el.classList.add(animClass);
                el.addEventListener('animationend', onEnd);
                setTimeout(finish, 1500); // Safety net if animationend never fires
            });
        }

        // Animate Dead: the new quantity and empowered art appear at the brightest moment of the flame
        async function empowerWithAnimation(elementId) {
            const spell = playSpell(elementId, 'anim-spectral');
            await wait(400);
            updateUI();
            await spell;
        }

        function spellMessage(spell, stack) {
            const them = opp ? opp.name : 'Your opponent';
            if (spell.type === 'double') {
                return spell.caster === 'you'
                    ? `You cast Animate Dead: your ${stack.name} is doubled and upgraded.`
                    : `${them} casts Animate Dead: their ${stack.name} is doubled and upgraded.`;
            }
            return spell.caster === 'you'
                ? `You cast Implosion: their ${stack.name} is destroyed.`
                : `${them} casts Implosion: your ${stack.name} is destroyed!`;
        }

        // End of a round: both armies are revealed, then both spells take effect
        async function animateResolve(m) {
            busy = true;
            setTargeting(false);
            if (me) me = { ...me, spell: null }; // The plan is cast now; its marker goes away
            fogOfWar = false;
            playerArmy = m.before.you.map(decorate);
            opponentArmy = m.before.opp.map(decorate);
            updateUI();
            await wait(REDUCE_MOTION ? 0 : 900);
            if (m.spells.length === 0) showToast('Neither side cast a spell this round.', 2500);
            for (const spell of m.spells) {
                const army = spell.side === 'you' ? playerArmy : opponentArmy;
                const stack = army[spell.index];
                if (!stack || stack.qty === 0) continue;
                const elementId = `${spell.side === 'you' ? 'player' : 'opp'}-card-${spell.index}`;
                showToast(spellMessage(spell, stack), 2600);
                if (spell.type === 'double') {
                    stack.qty *= 2;
                    stack.empowered = true;
                    await empowerWithAnimation(elementId);
                } else {
                    stack.qty = 0;
                    await playSpell(elementId, 'anim-void');
```

```
                    updateUI();
                }
                await wait(REDUCE_MOTION ? 0 : 600);
            }
            busy = false;
            updateUI();
        }

        // --- GAME OVER and the Crypt Fortune ---
        async function showFortune(tier, bet, winnings) {
            const multiplier = tier ? tier.multiplier : 1;
            els.fortune.className = 'fortune';
            els.fortuneHeadline.textContent = '';
            els.fortuneResult.textContent = '';
            els.fortune.hidden = false;
            if (!REDUCE_MOTION) {
                for (let step = 0, delay = 40; step < 14; step++, delay *= 1.15) {
                    els.fortuneReel.textContent = `×${FORTUNE_FACES[step % FORTUNE_FACES.length]}`;
                    await wait(delay);
                }
            }
            els.fortuneReel.textContent = `×${multiplier}`;
            if (tier) {
                els.fortune.classList.add(`tier-${tier.key}`);
                els.fortuneHeadline.textContent = `${tier.name}!`;
                els.fortuneResult.textContent = `Your ${winnings} gold winnings ×${multiplier} = ${winnings * multiplier} gold. You collect ${bet + winnings * multiplier} gold.`;
            } else {
                els.fortuneHeadline.textContent = 'Victory';
                els.fortuneResult.textContent = `No fortune this time. You collect ${bet + winnings} gold.`;
            }
            await wait(tier ? 3200 : 1500);
            els.fortune.hidden = true;
        }

        async function showGameOver(m) {
            busy = true;
            updateUI();
            const them = opp ? opp.name : 'Your opponent';
            const credit = markPaid(m.gameId);
            if (m.outcome === 'win') {
                if (credit) { gold += m.collected; saveGold(); }
                const tier = FORTUNE_TIERS.find(t => t.key === m.fortune) || null;
                await showFortune(tier, m.bet, m.winnings);
                showToast(tier
                    ? `${tier.name}! You beat ${them} ${m.you} to ${m.opp} and collect ${m.collected} gold.`
                    : `VICTORY! You beat ${them} ${m.you} to ${m.opp}. Your ${m.bet} gold bet comes back with ${m.winnings} gold winnings.`, 6000);
            } else if (m.outcome === 'loss') {
                showToast(`DEFEAT. ${them} wins ${m.opp} to ${m.you}. The Necropolis claims your bet.`, 6000);
            } else {
                if (credit) { gold += m.collected; saveGold(); }
                showToast(`DRAW at ${m.you} points each. Your bet is returned.`, 6000);
            }
            busy = false;
            updateUI();
        }

        // --- THE MAIN BUTTON: bet, draw the next round, or play again ---
        els.btnDraw.addEventListener('click', async () => {
            if (!me || busy || me.ready) return;
            if (phase === 'betting') {
                const bet = Number(els.betInput.value.trim());
                if (!Number.isInteger(bet) || bet < 1) { showToast('Place a bet of at least 1 gold to begin.'); return; }
                if (bet > gold) { showToast(`You only have ${gold} gold.`); return; }
                if (await sendAction({ type: 'bet', amount: bet })) {
                    gold -= bet;
                    saveGold();
                    updateUI();
                }
            } else if (phase === 'reveal') {
                await sendAction({ type: 'continue' });
            } else if (phase === 'finished') {
                await sendAction({ type: 'again' });
            }
        });

        els.betInput.addEventListener('keydown', (e) => {
            if (e.key === 'Enter') els.btnDraw.click();
        });

        // --- RENDER ---
        function updateDeck() {
            els.deckDisp.textContent = Math.max(0, deckCount);
            els.deckPile.classList.toggle('empty', deckCount <= 0);
        }
```

```
1693
1694        function drawLabel() {
1695            if (!me || phase === 'lobby' || phase === 'connecting') return 'Waiting for Opponent';
1696            if (phase === 'betting') return me.ready ? 'Waiting for Opponent' : 'Place Bet & Draw';
1697            if (phase === 'reveal') return me.ready ? 'Waiting for Opponent' : 'Draw Next Round';
1698            if (phase === 'finished') return me.ready ? 'Waiting for Opponent' : 'Play Again';
1699            return 'Draw Army';
1700        }
1701
1702        function opponentStatus() {
1703            if (!opp) return ['Not here yet', ''];
1704            if (!opp.connected) return ['Away', 'is-away'];
1705            if (phase === 'betting') return opp.betPlaced ? ['Bet placed', 'is-ready'] : ['Placing a bet…', ''];
1706            if (phase === 'spells') return opp.ready ? ['Ready', 'is-ready'] : opp.spellChosen ? ['Spell chosen…', ''] :
    ['Choosing a spell…', ''];
1707            if (phase === 'reveal') return opp.ready ? ['Ready for the next round', 'is-ready'] : ['Studying the table…',
    ''];
1708            if (phase === 'finished') return opp.ready ? ['Wants a rematch', 'is-ready'] : ['Game over', ''];
1709            return ['', ''];
1710        }
1711
1712        function updateUI() {
1713            els.roundDisp.textContent = `${Math.max(currentRound, 1)} / ${TOTAL_ROUNDS}`;
1714            els.potDisp.textContent = totalPot;
1715            els.goldDisp.textContent = gold;
1716            updateDeck();
1717
1718            els.oppName.textContent = opp ? opp.name : 'Waiting for an opponent';
1719            const [statusText, statusClass] = opponentStatus();
1720            els.oppStatus.textContent = statusText;
1721            els.oppStatus.className = `opp-status ${statusClass}`;
1722            els.oppStatus.hidden = !statusText;
1723            els.youName.textContent = me ? me.name : 'Your Army';
1724
1725            // Buttons state
1726            const waitingOnMe = Boolean(me) && !me.ready && !busy;
1727            els.btnDraw.textContent = drawLabel();
1728            els.btnDraw.disabled = !(waitingOnMe && ['betting', 'reveal', 'finished'].includes(phase));
1729            els.btnSpell.disabled = !canPlanSpell();
1730            els.btnSpell.textContent = spellLabel();
1731            els.btnResolve.disabled = !canPlanSpell();
1732            els.btnResolve.textContent = phase === 'spells' && me && me.ready ? 'Waiting for Opponent' : 'Ready';
1733            els.betInput.disabled = !(waitingOnMe && phase === 'betting');
1734
1735            // Strengths: your opponent's stays hidden until the round's end
1736            els.playerStrength.textContent = getArmyStrength(playerArmy);
1737            els.oppStrength.textContent = fogOfWar ? '???' : getArmyStrength(opponentArmy);
1738
1739            // Render Boards
1740            renderArmy(playerArmy, els.playerSlots, false);
1741            renderArmy(opponentArmy, els.oppSlots, true);
1742        }
1743
1744        // Updates the board in place so flips, spell effects and quantity changes animate
1745        function renderArmy(army, container, isOpponent) {
1746            if (container.children.length !== SLOT_COUNT) {
1747                container.innerHTML = '';
1748                // Always render 5 physical slots
1749                for (let i = 0; i < SLOT_COUNT; i++) {
1750                    const slot = document.createElement('div');
1751                    slot.className = 'card-slot';
1752                    slot.style.zIndex = SLOT_COUNT - i; // Left cards sit over right ones, so a pile's x2 badge is never c
    overed
1753                    slot.addEventListener('click', () => handleCardClick(isOpponent, i));
1754                    slot.addEventListener('keydown', (e) => {
1755                        if ((e.key === 'Enter' || e.key === ' ') && e.target.classList.contains('card-wrapper')) {
1756                            e.preventDefault();
1757                            handleCardClick(isOpponent, i);
1758                        }
1759                    });
1760                    container.appendChild(slot);
1761                }
1762            }
1763
1764            const hidden = isOpponent && fogOfWar;
1765            for (let i = 0; i < SLOT_COUNT; i++) {
1766                const slot = container.children[i];
1767                const stack = army[i];
1768                const alive = Boolean(stack && stack.qty > 0);
1769                const key = alive ? `${stack.id}:${stack.known ? 'k' : 'u'}` : '';
1770
1771                let revealing = false;
1772                if (slot.dataset.key !== key) {
1773                    // The same pile becoming known is a reveal: rebuild it face down, then flip it over
1774                    revealing = Boolean(alive && slot.dataset.key && slot.dataset.key.split(':')[0] === stack.id);
1775                    slot.dataset.key = key;
```

```
                    slot.innerHTML = alive
                        ? buildCardHTML(stack, i, isOpponent, { faceDown: revealing || hidden, settle: !revealing })
                        : '<div class="slot-empty"></div>';
                    if (revealing && !hidden) {
                        const card = slot.querySelector('.card');
                        requestAnimationFrame(() => requestAnimationFrame(() => card.classList.remove('face-down')));
                    }
                }

                // Planned spell marker (only you see your own plan)
                const plan = me && me.spell && phase === 'spells' && me.spell.index === i &&
                    ((me.spell.type === 'double' && !isOpponent) || (me.spell.type === 'implode' && isOpponent)) ? me.spel
l.type : null;
                slot.classList.toggle('plan-double', plan === 'double' && alive);
                slot.classList.toggle('plan-implode', plan === 'implode' && alive);
                let tag = slot.querySelector('.plan-tag');
                if (plan && alive && !tag) {
                    tag = document.createElement('div');
                    tag.className = 'plan-tag';
                    slot.appendChild(tag);
                }
                if (tag) {
                    if (plan && alive) tag.textContent = plan === 'double' ? 'Animate Dead' : 'Implosion';
                    else tag.remove();
                }

                if (!alive) continue;

                const cardEl = slot.querySelector('.card');
                if (!revealing) cardEl.classList.toggle('face-down', hidden);
                slot.querySelector('.card-wrapper').setAttribute('aria-label', hidden || !stack.known
                    ? 'Hidden enemy card'
                    : `${isOpponent ? 'Enemy' : 'Your'} ${stack.empowered ? 'upgraded ' : ''}level ${stack.level} ${stack.
name}, ${stackPoints(stack)} points`);

                const art = slot.querySelector('.card-art');
                if (art && stack.known && art.getAttribute('src') !== artSrc(stack, stack.empowered)) {
                    art.setAttribute('src', artSrc(stack, stack.empowered));
                }

                // Points pulse whenever they change (a matching card joins, or Animate Dead)
                const points = slot.querySelector('.card-points');
                if (points) {
                    const pointsText = String(stackPoints(stack));
                    const value = points.querySelector('.card-points-value');
                    if (value.textContent !== pointsText) {
                        value.textContent = pointsText;
                        pulse(points);
                    }
                }

                // Matching cards: a pile behind the card and an x2, x3 badge (single cards show neither)
                const wrapper = slot.querySelector('.card-wrapper');
                wrapper.classList.toggle('stack-2', stack.qty === 2);
                wrapper.classList.toggle('stack-3', stack.qty >= 3);
                const count = slot.querySelector('.card-count');
                const countText = `x${stack.qty}`;
                count.hidden = stack.qty < 2;
                if (count.textContent !== countText) {
                    count.textContent = countText;
                    if (!count.hidden) pulse(count);
                }
            }
        }

        // Replays the little bump animation on a badge whose number just changed
        function pulse(el) {
            el.classList.remove('badge-bump');
            void el.offsetWidth;
            el.classList.add('badge-bump');
        }

        // The face of a card: creature art, name and the points it gives (blank while the card is unknown)
        function cardFrontHTML(stack) {
            if (!stack.name) return '<div class="card-face card-front card-front-unknown"></div>';
            // Creature art fills the card; units without art keep the skull sigil
            const artHtml = stack.art
                ? `<img class="card-art" src="${artSrc(stack, stack.empowered)}" alt="" draggable="false">`
                : '';

            return `
                <div class="card-face card-front ${stack.art ? 'has-art' : ''}" style="--lv: ${stack.color};">
                    ${artHtml}
                    <div class="card-name">${stack.name}</div>
                    <div class="card-sigil">${stack.art ? '' : '☠'}</div>
                    <div class="card-foot">
```

```
                        <span class="card-points"><span class="card-points-value">${stackPoints(stack)}</span><small>pts</small></span>
                    </div>
                </div>
            `;
        }

        function buildCardHTML(stack, index, isOpponent, { faceDown, settle }) {
            const prefix = isOpponent ? 'opp' : 'player';

            // Random positioning applied to the wrapper inside the slot
            const transformStyle = `transform: translate(${stack.tx}px, ${stack.ty}px) rotate(${stack.rot}deg);`;

            return `
                <div class="card-wrapper" style="${transformStyle}" tabindex="0" role="button">
                    <div id="${prefix}-card-${index}" class="card-fx ${settle ? 'animate-land' : ''}">
                        <div class="card ${faceDown ? 'face-down' : ''}">
                            ${cardFrontHTML(stack)}
                            <div class="card-face card-back"></div>
                        </div>
                        <div class="card-count" hidden></div>
                    </div>
                </div>
            `;
        }

        // --- RUN: rejoin this tab's seat if there is one, otherwise ask for a name ---
        (async function start() {
            updateUI();
            if (location.protocol === 'file:') {
                // Opened straight from the folder: the game only works through the server
                els.joinError.textContent = 'This file can\'t be opened directly. Close it, double-click start-server.bat (Mac: start-server.command) in the same folder, and the game opens in your browser.';
                els.joinName.disabled = true;
                els.joinForm.querySelector('button').disabled = true;
                els.join.hidden = false;
                return;
            }
            const savedName = store.get('localStorage', 'necropolis-duel-name') || '';
            const savedToken = store.get('sessionStorage', 'necropolis-duel-token');
            els.joinName.value = savedName;
            if (savedName && savedToken) {
                try {
                    await joinTable(savedName, savedToken);
                    return;
                } catch (err) {
                    els.joinError.textContent = err.message;
                }
            }
            els.join.hidden = false;
            els.joinName.focus();
        })();
    </script>
</body>
</html>
```

build.py

Build script: embeds pictures and styles / აწყობის სკრიპტი · 71 lines · SHA-256 55f5dea4855c6b39e75c434cf81a299621b095022c2c30d9262760cdd2d9120e

```
"""Bundles a game page into one self-contained HTML file.

- Card pictures from img/ are embedded as data URIs (the build:card-art block),
  so the page shows them even when opened on its own, without the img/ folder.
- Tailwind styles are precompiled into the page (the build:tailwind block), so
  there is no Tailwind CDN script. This step needs Node.js (it runs npx).

Run it after changing the card art or the Tailwind classes in a page:
    python3 build.py                        # index.html: pictures and styles
    python3 build.py multiplayer/client.html  # the two-player page
    python3 build.py --art-only             # pictures only, no Node.js needed
"""
import base64
import json
import re
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

HERE = Path(__file__).parent
TAILWIND = 'tailwindcss@3.4.17'


def replace_block(html, name, body):
    """Swaps the content between <!-- build:name ... --> and <!-- /build:name -->."""
    pattern = re.compile(rf'(<!-- build:{name}\b[^>]*-->\n)(.*?)(\n[ \t]*<!-- /build:{name} -->)', re.S)
    html, count = pattern.subn(lambda m: m.group(1) + body + m.group(3), html)
    if count != 1:
        raise SystemExit(f'build:{name} block not found in the page')
    return html


def card_art_block():
    entries = []
    for picture in sorted((HERE / 'img').glob('*.webp')):
        data = base64.b64encode(picture.read_bytes()).decode('ascii')
        entries.append(f'        {json.dumps(picture.stem)}: "{data}"')
    return '    <script id="card-art">\n    const CARD_ART = {\n' + ',\n'.join(entries) + '\n    };\n    </script>'


def tailwind_block(html):
    npx = shutil.which('npx')
    if not npx:
        raise SystemExit('Node.js (npx) is needed to rebuild the styles; use --art-only to skip them')
    # Scan only the real markup and scripts, not the embedded pictures or the old styles
    scan = replace_block(replace_block(html, 'card-art', ''), 'tailwind', '')
    with tempfile.TemporaryDirectory() as tmp:
        tmp = Path(tmp)
        (tmp / 'page.html').write_text(scan, encoding='utf-8')
        (tmp / 'in.css').write_text('@tailwind base;\n@tailwind components;\n@tailwind utilities;\n')
        subprocess.run([npx, '--yes', TAILWIND, '-i', str(tmp / 'in.css'), '-o', str(tmp / 'out.css'),
                        '--content', str(tmp / 'page.html'), '--minify'], check=True)
        css = (tmp / 'out.css').read_text(encoding='utf-8').strip()
    return f'    <style id="tailwind">{css}</style>'


def main():
    pages = [arg for arg in sys.argv[1:] if not arg.startswith('--')]
    page = Path(pages[0]) if pages else HERE / 'index.html'
    html = page.read_text(encoding='utf-8')
    html = replace_block(html, 'card-art', card_art_block())
    if '--art-only' not in sys.argv:
        html = replace_block(html, 'tailwind', tailwind_block(html))
    page.write_text(html, encoding='utf-8')
    print(f'built {page} ({page.stat().st_size // 1024} KB)')


if __name__ == '__main__':
    main()
```

art-source/cut_figures.py

Art step 1: cuts creatures out of the source images / ილუსტრაციები, ნაბიჯი 1 · 76 lines · SHA-256 9b57b4bfb9b02cbe5a1ac61d417861bb90eac2abdcf0a1be206f3dc4c4e77810

```
"""Step 1: cut every creature out of its source image (transparent background).

Each source image shows two variants side by side: the weaker base unit on the
left and its stronger form on the right (shown only after Animate Dead).
Results go to cutouts/<unit>.png and cutouts/<unit>-empowered.png; they can be
touched up by hand before running make_card_art.py.

Usage: pip install "rembg[cpu]" && python3 cut_figures.py
"""
from pathlib import Path

import numpy as np
from PIL import Image
from rembg import new_session, remove

HERE = Path(__file__).parent
OUT = HERE / 'cutouts'

# Figure bounds (left, top, right, bottom) in source pixels: base, empowered
FIGURES = {
    'skeleton':    ('skeleton.png',     (64, 118, 250, 452),  (294, 100, 478, 452)),
    'zombie':      ('zombie.png',       (64, 88, 283, 452),   (284, 82, 474, 452)),
    'wight':       ('wight.webp',       (80, 24, 226, 462),   (280, 24, 448, 476)),
    'vampire':     ('vampire.png',      (96, 108, 264, 480),  (255, 108, 462, 480)),
    'lich':        ('lich.png',         (24, 58, 264, 456),   (256, 24, 490, 456)),
    'dark-knight': ('dark-knight.webp', (25, 205, 505, 790),  (520, 255, 985, 805)),
    'bone-dragon': ('bone-dragon.webp', (30, 200, 492, 828),  (498, 195, 1000, 835)),
}


def keep_main_shapes(alpha, min_share=0.04):
    """Drops specks and slivers of the neighbouring figure; keeps the creature and its large props."""
    solid = alpha > 96
    h, w = solid.shape
    labels = np.zeros((h, w), dtype=np.int32)
    sizes = [0]
    for y0, x0 in zip(*np.nonzero(solid)):
        if labels[y0, x0]:
            continue
        label = len(sizes)
        stack, count = [(y0, x0)], 0
        labels[y0, x0] = label
        while stack:
            y, x = stack.pop()
            count += 1
            for ny, nx in ((y + 1, x), (y - 1, x), (y, x + 1), (y, x - 1)):
                if 0 <= ny < h and 0 <= nx < w and solid[ny, nx] and not labels[ny, nx]:
                    labels[ny, nx] = label
                    stack.append((ny, nx))
        sizes.append(count)
    if len(sizes) == 1:
        return alpha
    biggest = max(sizes)
    keep = np.array([False] + [s >= biggest * min_share for s in sizes[1:]])
    # Soft edge pixels follow the shape they border
    grown = keep[labels]
    for _ in range(2):
        grown = grown | np.roll(grown, 1, 0) | np.roll(grown, -1, 0) | np.roll(grown, 1, 1) | np.roll(grown, -1, 1)
    return np.where(grown, alpha, 0).astype(np.uint8)


def main():
    OUT.mkdir(exist_ok=True)
    session = new_session('birefnet-general-lite')
    for unit, (filename, base_box, empowered_box) in FIGURES.items():
        src = Image.open(HERE / filename).convert('RGB')
        for suffix, box in (('', base_box), ('-empowered', empowered_box)):
            cut = remove(src.crop(box), session=session).convert('RGBA')
            rgba = np.array(cut)
            rgba[..., 3] = keep_main_shapes(rgba[..., 3])
            Image.fromarray(rgba).save(OUT / f'{unit}{suffix}.png', optimize=True)
            print('cut', unit + suffix)


if __name__ == '__main__':
    main()
```

art-source/make_card_art.py

Art step 2: draws the neon card pictures / ილუსტრაციები, ნაბიჯი 2 · 196 lines · SHA-256 176a362b0bcf52db137c54fc8a719093ceafb24f73ab0fd157ead55bc3d6bedc

```
"""Step 2: turn the creature cutouts into one matching set of neon cards.

The source art comes in different styles (painted, inked, pixel art), so every
creature is redrawn the same way: a dark figure lit in its level's neon colour,
with a glowing outline, standing in a neon crypt arch over a glowing grid floor.
The empowered form (shown after Animate Dead) gets a green Animate Dead aura,
green arch and drifting sparks; its outline keeps the level colour.

Usage: python3 make_card_art.py   (needs Pillow and numpy; reads cutouts/, writes ../img/)
Then run ../build.py to embed the new pictures into the game pages.
"""
import random
from pathlib import Path

import numpy as np
from PIL import Image, ImageDraw, ImageFilter

HERE = Path(__file__).parent
CUTOUTS = HERE / 'cutouts'
OUT = HERE.parent / 'img'

# Level colours: keep these the same as UNIT_DATA in the game pages
UNITS = {
    'skeleton': '#cfd8ff',     # Lv 1, ice white
    'zombie': '#4dff88',       # Lv 2, green
    'wight': '#38d6ff',        # Lv 3, sky blue
    'vampire': '#c77dff',      # Lv 4, violet
    'lich': '#ff9a3c',         # Lv 5, orange
    'dark-knight': '#ff4d6d',  # Lv 6, red
    'bone-dragon': '#ffe047',  # Lv 7, gold
}
ANIMATE = '#4dff88'            # Animate Dead green, for empowered cards

W, H = 260, 360                # Saved size: 2x the largest card (130 x 180)
SS = 2                         # Drawn at 2x and scaled down, for smooth lines
FIGURE_TOP = 0.15              # Keeps heads clear of the name banner
FEET = 0.93                    # Figures stand here
HORIZON = 0.74                 # Where the back wall meets the grid floor
INK = np.array([7, 8, 18], dtype=np.float64)


def rgb(hex_colour):
    return np.array([int(hex_colour[i:i + 2], 16) for i in (1, 3, 5)], dtype=np.float64)


def mix(a, b, t):
    return a + (b - a) * t


def glow_layer(draw_fn, size, blur):
    """Draws white lines with draw_fn and returns (sharp core, soft glow) as 0-1 arrays."""
    layer = Image.new('L', size, 0)
    draw_fn(ImageDraw.Draw(layer))
    core = np.asarray(layer, dtype=np.float64) / 255
    soft = np.asarray(layer.filter(ImageFilter.GaussianBlur(blur)), dtype=np.float64) / 255
    return core, soft


def add_light(canvas, amount, colour):
    """Adds coloured light (screen-like, never past white)."""
    light = amount[..., None] * colour[None, None, :]
    return 255 - (255 - canvas) * (1 - np.clip(light / 255, 0, 1))


def fit_figure(cutout, cw, ch):
    box_w, box_h = cw * 0.84, ch * (FEET - FIGURE_TOP)
    fit = min(box_w / cutout.width, box_h / cutout.height)
    size = (max(1, round(cutout.width * fit)), max(1, round(cutout.height * fit)))
    small = cutout.convert('RGBa').resize(size, Image.LANCZOS).convert('RGBA')
    return small, ((cw - size[0]) // 2, round(ch * FEET) - size[1])


def make_card(unit, empowered):
    cw, ch = W * SS, H * SS
    colour = rgb(UNITS[unit])
    accent = rgb(ANIMATE) if empowered else colour
    rng = random.Random(f'{unit}-{empowered}')

    cutout = Image.open(CUTOUTS / f"{unit}{'-empowered' if empowered else ''}.png").convert('RGBA')
    figure, (fx, fy) = fit_figure(cutout, cw, ch)
    fw, fh = figure.size
```

```
    cx = fx + fw / 2

    # Backdrop: deep ink, a little lighter toward the floor, with a soft pool of the level colour
    yy, xx = np.mgrid[0:ch, 0:cw].astype(np.float64)
    canvas = np.zeros((ch, cw, 3)) + mix(INK, INK * 1.6, (yy / ch)[..., None])
    pool = np.exp(-(((xx - cx) / (cw * 0.42)) ** 2 + ((yy - ch * 0.55) / (ch * 0.38)) ** 2))
    canvas = add_light(canvas, pool * 0.16, colour)

    horizon = ch * HORIZON
    vanish_y = horizon - ch * 0.22

    # Grid floor in perspective, fading toward the horizon
    def floor(d):
        for k in range(1, 9):
            t = (k / 8) ** 1.8
            y = horizon + (ch - horizon) * t
            d.line((0, y, cw, y), fill=int(70 + 150 * t), width=SS)
        for k in range(-7, 8):
            x_bottom = cx + k * cw * 0.16
            x_top = cx + (x_bottom - cx) * (horizon - vanish_y) / (ch - vanish_y)
            d.line((x_top, horizon, x_bottom, ch), fill=150, width=SS)
    core, soft = glow_layer(floor, (cw, ch), 3 * SS)
    depth = np.clip((yy - horizon) / (ch - horizon), 0, 1)
    canvas = add_light(canvas, (core * 0.75 + soft * 0.55) * (0.3 + 0.7 * depth), colour)

    # Neon crypt arch around the creature, and the horizon line
    half = min(cw / 2 - 8 * SS, fw / 2 + 16 * SS)
    arch_top = max(ch * 0.07, fy - 14 * SS)

    def arch(inset):
        def draw(d):
            x0, x1 = cx - half + inset, cx + half - inset
            r = (x1 - x0) / 2
            top = arch_top + inset
            d.arc((x0, top, x1, top + 2 * r), 180, 360, fill=255, width=2 * SS)
            d.line((x0, top + r, x0, horizon), fill=255, width=2 * SS)
            d.line((x1, top + r, x1, horizon), fill=255, width=2 * SS)
        return draw
    core, soft = glow_layer(arch(0), (cw, ch), 6 * SS)
    canvas = add_light(canvas, soft * 0.9, accent)
    canvas = add_light(canvas, core * 0.75, mix(accent, np.full(3, 255.0), 0.45))
    if empowered:
        core, soft = glow_layer(arch(9 * SS), (cw, ch), 4 * SS)
        canvas = add_light(canvas, soft * 0.5 + core * 0.45, accent)
    core, soft = glow_layer(lambda d: d.line((0, horizon, cw, horizon), fill=255, width=SS), (cw, ch), 5 * SS)
    canvas = add_light(canvas, soft * 0.7 + core * 0.6, colour)

    # Where the figure is (soft edge kept for anti-aliasing)
    alpha_img = Image.new('L', (cw, ch), 0)
    alpha_img.paste(figure.getchannel('A'), (fx, fy))
    alpha = np.asarray(alpha_img, dtype=np.float64) / 255

    # Empowered: a green Animate Dead aura behind the creature
    if empowered:
        aura = np.asarray(alpha_img.filter(ImageFilter.MaxFilter(9)).filter(ImageFilter.GaussianBlur(16 * SS)), dtype=np.f
loat64) / 255
        canvas = add_light(canvas, np.clip(aura * 1.6, 0, 1) * 0.6, accent)

    # Figure: its own shading mapped onto one dark-to-neon ramp, so every creature matches
    src = np.zeros((ch, cw, 3))
    src[fy:fy + fh, fx:fx + fw] = np.asarray(figure, dtype=np.float64)[..., :3]
    lum = src @ np.array([0.299, 0.587, 0.114]) / 255
    inside = alpha > 0.5
    lo, hi = np.percentile(lum[inside], 3), np.percentile(lum[inside], 97)
    lum = np.clip((lum - lo) / max(hi - lo, 1e-3), 0, 1) ** 1.1
    top = 0.62 if empowered else 0.5
    stops = [(0.0, INK), (0.45, colour * 0.36), (0.78, colour * 0.85), (1.0, mix(colour, np.full(3, 255.0), top))]
    shade = np.zeros_like(src)
    for (t0, c0), (t1, c1) in zip(stops, stops[1:]):
        part = np.clip((lum - t0) / (t1 - t0), 0, 1)[..., None]
        band = ((lum >= t0) & (lum <= t1))[..., None]
        shade = np.where(band, c0 + (c1 - c0) * part, shade)

    # Back light: the figure's edge catches the neon
    edge_in = alpha - np.asarray(alpha_img.filter(ImageFilter.MinFilter(7)).filter(ImageFilter.GaussianBlur(2 * SS)), dtyp
e=np.float64) / 255
    shade = add_light(shade, np.clip(edge_in, 0, 1) * 0.55, accent)
    canvas = canvas * (1 - alpha[..., None]) + shade * alpha[..., None]

    # Neon outline around the figure
    ring_img = alpha_img.filter(ImageFilter.MaxFilter(5))
    ring = np.clip(np.asarray(ring_img, dtype=np.float64) / 255 - alpha, 0, 1)
    ring_soft = np.asarray(Image.fromarray((ring * 255).astype(np.uint8)).filter(ImageFilter.GaussianBlur(5 * SS)), dtype=
np.float64) / 255
    canvas = add_light(canvas, np.clip(ring_soft * 1.8, 0, 1) * 0.8, colour)
    canvas = add_light(canvas, ring * 0.9, mix(colour, np.full(3, 255.0), 0.5))
```

```

    # Empowered: drifting green sparks
    if empowered:
        def sparks(d):
            for _ in range(22):
                x = rng.uniform(cw * 0.08, cw * 0.92)
                y = rng.uniform(ch * 0.12, horizon)
                r = rng.uniform(1.0, 2.4) * SS
                d.ellipse((x - r, y - r, x + r, y + r), fill=255)
        core, soft = glow_layer(sparks, (cw, ch), 3 * SS)
        canvas = add_light(canvas, soft * 0.9 + core * 0.8, mix(accent, np.full(3, 255.0), 0.3))

    # Darker corners and a dark band at the top, so the name stays readable
    vignette = np.clip(1 - 0.55 * (((xx - cw / 2) / (cw * 0.75)) ** 2 + ((yy - ch / 2) / (ch * 0.75)) ** 2), 0.35, 1)
    canvas *= vignette[..., None]
    canvas *= np.clip(0.55 + yy / (ch * 0.16) * 0.45, 0.55, 1)[..., None]

    art = Image.fromarray(np.clip(canvas, 0, 255).astype(np.uint8))
    return art.resize((W, H), Image.LANCZOS)


def main():
    OUT.mkdir(exist_ok=True)
    for unit in UNITS:
        for empowered in (False, True):
            name = f"{unit}{'-empowered' if empowered else ''}.webp"
            make_card(unit, empowered).save(OUT / name, quality=82, method=6)
            print('wrote', name, (OUT / name).stat().st_size // 1024, 'KB')


if __name__ == '__main__':
    main()
```

multiplayer/start-server.bat

Windows start script / Windows-ის გაშვების სკრიპტი · 22 lines · SHA-256 028d76101b167b99cb7e933b24239d34279fc3f274f37b057a106db623e4ae7f

```
@echo off
title Necropolis Casino server
rem Starts the Necropolis Casino Duel server. Keep this window open while you play.
cd /d "%~dp0"
if not exist "server.js" goto not_unzipped
where node >nul 2>nul || goto no_node
node server.js
pause
exit /b

:not_unzipped
echo server.js is not next to this file, so the zip has not been unzipped yet.
echo Right-click necropolis-casino-duel.zip, choose "Extract All", open the
echo extracted folder and double-click start-server.bat in there.
pause
exit /b 1

:no_node
echo Node.js was not found. Install the LTS version from https://nodejs.org
echo then restart the PC and double-click start-server.bat again.
pause
exit /b 1
```

multiplayer/start-server.command

macOS start script / macOS-ის გაშვების სკრიპტი · 6 lines · SHA-256 b5b86e3a1e3bfcbc34fa95b27c24ed714440b849a51af5d53fe0d3d81cc77751

```
#!/bin/sh
# Starts the Necropolis Casino Duel server (macOS: double-click this file). Keep the window open while you play.
cd "$(dirname "$0")" || exit 1
if [ ! -f server.js ]; then echo "server.js is not next to this file. Unzip the download first, then double-click start-server.command in the unzipped folder."; read -r _; exit 1; fi
if ! command -v node >/dev/null 2>&1; then echo "Node.js is not installed. Get the LTS version from https://nodejs.org and run this again."; read -r _; exit 1; fi
node server.js
```